Interactive Query over Multiple Datasets

Abstract

The interactive query is a representative approach for multi-criteria decision-making. It enables systems to interact with users to learn their preferences and then return tuples accordingly. However, existing interactive algorithms focus merely on single-dataset scenarios, where all tuples are placed in one dataset and the goal is to identify the user's preferred tuples from this dataset. In many practical applications, tuples are grouped into multiple independent datasets, and users desire to find their favorite tuple within each individual one. Motivated by this, we formalize a novel problem: Interactive Query over Multiple Datasets (IMD).

To solve the IMD problem, we exploit the underlying consistency of user preference across datasets and develop algorithms at two levels of sharing. First, we propose the *shared utility range* (SUR) mechanism, which propagates learned preference across datasets. Equipped with this mechanism, we derive four variants of existing interactive algorithms: SUR-HD-PI, SUR-RH, SUR-UH-Random, and SUR-UH-Simplex. Second, we introduce the *shared questions* mechanism, which enables each question to be informative for multiple datasets simultaneously. To further address the scalability bottleneck, we develop a localized strategy that first detects promising candidate tuples from each dataset and then selects shared questions within these localized ranges. By combining the shared questions mechanism with this localized strategy, we propose algorithm SharedQ. Experiments on synthetic and real datasets show that our algorithms reduce the number of questions by 50% to 90% while retaining high computational efficiency.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

ACM Reference Format:

. 2018. Interactive Query over Multiple Datasets. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Multi-criteria decision-making [8, 10, 16, 43, 48] aims to help users identify desired tuples from large-scale databases. It has been widely used in real-world applications, e.g., apartment rentals, job seeking, and car purchases. One representative approach for multi-criteria decision-making is the interactive query [5, 19, 22, 25, 30], which aims to interact with users to learn their preferences and then return tuples based on the learned preferences.

Table 1: Car Dataset

Car	Horsepower (HP)	Safety Score (out of 100)	Brand
Car 1	120	95	Audi
Car 2	180	85	Audi
Car 3	240	70	Audi
Car 4	150	100	Ford
Car 5	200	95	Ford

In general, the interactive query models the user's preference through a *linear utility function* $f_{\boldsymbol{u}}(\boldsymbol{p}) = \boldsymbol{u} \cdot \boldsymbol{p}$, where $\boldsymbol{p}$ denotes a tuple and $\boldsymbol{u}$ is a multi-dimensional vector, called *utility vector* [9, 28, 46, 47]. Each dimension of $\boldsymbol{u}$ indicates the user's hidden preference weight for the corresponding dimension of $\boldsymbol{p}$, and a larger function score indicates that the tuple is more preferred by the user. To learn the user's preference, i.e., the user's specific $\boldsymbol{u}$, the interactive query asks the user a sequence of pairwise comparison questions [38, 39]: *Do you prefer tuple $\boldsymbol{p}$ or tuple $\boldsymbol{q}$?* Each feedback induces a constraint on $\boldsymbol{u}$. For example, if the user prefers $\boldsymbol{p}$ to $\boldsymbol{q}$, then $f_{\boldsymbol{u}}(\boldsymbol{p}) > f_{\boldsymbol{u}}(\boldsymbol{q})$, i.e., $\boldsymbol{u} \cdot (\boldsymbol{p} - \boldsymbol{q}) > 0$. By accumulating such constraints, the feasible range of $\boldsymbol{u}$ is gradually narrowed. When the range becomes sufficiently small, the user's preference, i.e., the user's $\boldsymbol{u}$, can be inferred, and consequently, tuples can be returned accordingly.

Consider Table 1 as an example. Each car is associated with two dimensions: horsepower and safety score. Suppose that Alice wants to purchase a car. The system interacts with Alice through several car pair comparisons, e.g., comparing Car 1 and Car 2. Based on Alice's feedback, the system gradually learns her $\boldsymbol{u} = (u[1], u[2])$ that captures her trade-off between horsepower and safety. Guided by this learned $\boldsymbol{u}$, the system returns cars with high scores.

Although promising, existing studies on the interactive queries mainly focus on the *single-dataset* setting, i.e., all tuples are placed in one dataset and the goal is to identify the user's preferred tuples from this dataset. However, such a setting may not fully capture the complexities of real-world scenarios. In many real-world scenarios, tuples are naturally organized into multiple groups [12, 26, 44, 45]. Each group corresponds to a diversity guarantee, a fairness constraint, an independent choice space, etc., and the user expects preferred tuples from each group rather than only from the union of all tuples. If all tuples are merged into one dataset, the returned tuples may concentrate on only a few dominant groups, leaving other required groups uncovered.

For example, Alice may want to receive one car from each brand for further comparison, as shown in Table 1. Although cars from different brands can be evaluated by horsepower and safety, each brand forms a separate set for diversity. If all cars are merged into one dataset, the returned cars may come from only a few dominant brands, failing to provide a diverse shortlist. Similarly, a school may need to select representative students from different groups, such as one male student and one female student, for fairness. Although all students can be evaluated by the same criteria, such as academic performance, leadership, and communication skills, the selection must

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

be conducted within each group separately. Moreover, a company may need to hire one candidate for each position, e.g., a database engineer, a frontend engineer, and a product manager. Each position defines a separate candidate set because candidates compete within the same role. Although the company may evaluate all candidates using similar criteria, such as experience, communication skills, and growth potential, a strong candidate for one position cannot replace the required hire for another position. Thus, the system should return preferred candidates from each position-specific dataset.

Motivated by this, we formally define a new problem *Interactive Query over Multiple Datasets (IMD).* Given multiple datasets, the goal is to interactively identify the user's favorite tuple within each individual dataset, rather than returning favorite tuples from their union. At the same time, the number of questions asked to the user should be minimized, since excessive interactions may lead to user fatigue and degrade the user experience. To the best of our knowledge, we are the first to formulate and study the IMD problem.

To solve the IMD problem, a straightforward idea is to sequentially run existing interactive algorithms [39, 42] over these datasets. However, it suffers from an efficiency flaw since the system treats the datasets independently and learns the user's preference from scratch for each of them. As shown in prior work [39, 42], any interactive algorithm requires $\Omega(\log_2 n)$ rounds for a dataset of size n. Thus, for m datasets, the total number of questions is lower bounded by $\Omega\left(\sum_{i=1}^{m} \log_2 n_i\right) = \Omega\left(\log_2 \prod_{i=1}^{m} n_i\right)$, where n_i is the size of the i-th dataset. This interaction cost can be prohibitive in practice.

Since treating datasets independently by existing algorithms comes with issues, our focus turns to sharing. The key observation is that although the favorite tuple must be identified within each dataset, all datasets are queried for the same user, and thus, are governed by the same utility vector. This creates an opportunity for sharing: information from one dataset can be shared with the others. Following this idea, we develop algorithms at two levels of sharing.

In the first level, we propose the *shared utility range* mechanism to share the information of learned preference across datasets. Instead of restarting the interaction from scratch for every dataset, we utilize the constraints implied by the user's feedback during interaction. We maintain a global *utility range* that records the feasible range of $\boldsymbol{u}$ induced by these constraints. When processing a new dataset, this shared utility range $\mathcal{R}$ is directly used to filter tuples that cannot be the user's favorite under any feasible utility vector in $\mathcal{R}$. In this way, user feedback collected from earlier datasets is shared with later datasets, avoiding redundant questions. Equipped with this mechanism, we propose 4 variants of the existing algorithms: SUR-HD-PI, SUR-RH, SUR-UH-Random, and SUR-UH-Simplex. However, sharing the utility range only still follows a sequential paradigm: the system asks questions for one dataset, updates the utility range, and then moves to another dataset.

In the second level, to further reduce interaction cost, we introduce the *shared questions* mechanism. The idea is to ask a question that is simultaneously useful for multiple datasets, thereby compressing multiple dataset-specific interactions into one shared interaction. Nevertheless, selecting effective shared questions is challenging because datasets may vary in size and distribution. A question that is highly informative for one dataset may be useless for another. Therefore, the system needs to find a balanced question that benefits as many datasets as possible. To this end, we formalize this shared question selection problem. Based on this formulation, we propose a heuristic method to select effective shared questions.

However, when the number of datasets or tuples becomes large, even the heuristic still faces two computational bottlenecks. On the one hand, the system must maintain up-to-date information for all datasets, which can be expensive; on the other hand, based on such maintained information, the system needs to evaluate many candidate questions over multiple datasets, which introduces high latency. To address these bottlenecks, we propose a localized strategy. It involves two phases. First, it identifies a small set of promising candidate tuples from each dataset based on the currently estimated user preference. Then, it selects and asks shared questions only within these identified tuples. By restricting to the most relevant portions of the datasets, it substantially reduces the computational cost while still keeping the number of questions small. Combining the shared questions mechanism and the localized strategy, we propose algorithm SharedQ.

Contributions. Our main contributions are as follows:

- To the best of our knowledge, we are the first to propose the *Interactive Query over Multiple Datasets (IMD)* problem, which aims to interactively find a user's favorite tuple in each given dataset. We also show a lower bound $\Omega(\log_2 N)$ on the number of questions asked during the interaction, where N is the maximum value among all dataset sizes.
- We augment algorithms UH-Random [42], UH-Simplex [42], RH [39], and HD-PI [39] with the *shared utility range* (SUR) mechanism. It reduces the number of questions asked to the user by transferring learned information across datasets.
- We develop SharedQ, a concurrent interactive exploration algorithm that integrates a shared questions mechanism across multiple datasets to reduce the number of questions asked to the user.
- Comprehensive experiments confirm the superiority of our algorithms over existing ones. For instance, algorithm SharedQ cuts down the number of questions by 50% to 90% and achieves substantial improvements in execution time under typical settings.

The rest of the paper is structured as follows. We review related work in Section 2 and formally define the problem in Section 3. The baselines (HD-PI, RH, UH-Random, and UH-Simplex), algorithms with the shared utility range mechanism (SUR-HD-PI, SUR-RH, SUR-UH-Random, SUR-UH-Simplex), and an algorithm with the shared question mechanism (SharedQ) are presented in Sections 4, 5, and 6, respectively. Experimental results are presented and analyzed in Section 7. Conclusion is shown in Section 8.

2 Related Work

Existing studies have introduced user interaction to mitigate the *multi-criteria decision-making* problem over a single dataset.

The interactive similarity query seeks to retrieve tuples that are close to a user's latent query tuple by iteratively learning both the latent query and the distance function [6, 7, 32]. Intuitively, it presents many candidate tuples to the user and asks for their relevance scores. Based on the user feedback, they update the estimated query tuple or distance function accordingly. Although this paradigm is flexible, it typically relies on extensive explicit feedback, requiring users to examine and score a large number of tuples. Such a heavy interaction burden makes them difficult to apply in practical scenarios.

To reduce the interaction burden, the interactive regret-minimizing query replaces fine-grained relevance scoring with simpler preference elicitation [27]. It returns a small set of tuples whose *regret ratio*, a criterion for evaluating tuples, is below a specified threshold. In each interactive round, the system presents several tuples, and the user only needs to select the most preferred one. According to the user's selection, the system shrinks the feasible region of the user's preference. When the feasible region is sufficiently small, the desired tuples can be returned. This interaction model is substantially lighter than asking the user to score many tuples. However, it displays *fake tuples* to the user during interaction, which are artificially constructed rather than selected from the database. This may create unrealistic tuples (e.g., a house with 200 square meters and costs only 100 dollars). To address this issue, Xie et al. proposed the *strongly truthful* regret-minimizing interactive query, which restricts all displayed objects to real tuples from the database [42].

Wang et al. propose to return one of the top-k tuples by interacting with the user [39]. They partition the space of possible user preferences into equivalence regions, where all preferences in the same region correspond to the same representative tuples. The interaction process is then formulated as a region-identification problem: each question is selected to distinguish among candidate regions, and the user's feedback progressively eliminates regions that are inconsistent with the observed feedback. Once the target region is identified, the system returns the corresponding representative tuples. Further developments can be seen in [37, 40], which explore the integration of different types of dimensions into the interaction.

In the domain of machine learning, our problem is closely related to *learning to rank* [13, 14, 17, 18, 23, 24]. It aims to infer the ranking of tuples through user interaction. However, most existing algorithms [14, 23, 24] rely solely on pairwise tuple preferences (i.e., tuple $\boldsymbol{p}$ is preferred over tuple $\boldsymbol{q}$) and neglect the internal relationships of dimensions (i.e., the dimension $p[1]$ of $\boldsymbol{p}$ is better than that $q[2]$ of $\boldsymbol{q}$). For instance, consider two tuples that differ only in price: one priced at \$200 and the other at \$500. If the internal relationship is considered, it can be inferred that the \$200 tuple is likely to be preferred without asking questions, given that a lower price is generally more desirable. [17] accounts for tuple interrelations. Nevertheless, since it focuses on the ranking of all tuples, it often involves redundant questions that contribute little to returning tuples. For instance, assume that Alice prefers tuple $\boldsymbol{p}_1$ to $\boldsymbol{p}_2$ and $\boldsymbol{p}_3$. We may not care about her preference between $\boldsymbol{p}_2$ and $\boldsymbol{p}_3$ when aiming to identify representative tuples. But these extra comparisons might be helpful in [17].

As for all the aforementioned algorithms, as discussed in Section 1, they mainly focus on a single dataset, assuming that all interactions are conducted within one fixed candidate space. This assumption may not hold in practice, as users often need to query across multiple datasets simultaneously. Consequently, existing frameworks cannot be applied to our setting off the shelf. To bridge this gap, we formulate the IMD problem and develop algorithms for interactive query processing over multiple datasets.

3 Problem Definition

Dataset. The input is a set of m datasets $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$. Each dataset $\mathcal{D}_i \in \mathrm{D}$ contains n_i tuples with the same d dimensions.

Table 2: Dataset

$\boldsymbol{p}$	$p[1]$	$p[2]$	$f_{\boldsymbol{u}}(\boldsymbol{p})$
$\boldsymbol{p}_1$	0.1	0.5	0.34
$\boldsymbol{p}_2$	0.3	0.4	0.36
$\boldsymbol{p}_3$	0.3	0.3	0.30
$\boldsymbol{p}_4$	0.5	0.2	0.32
$\boldsymbol{p}_5$	0.6	0.1	0.30

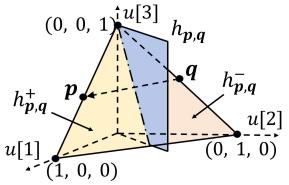


Figure 1: Utility Space in $\mathbb{R}^3$

We regard each tuple as a point in a d-dimensional space $\mathbb{R}^d$, and in the following, use the terms *tuple* and *point* interchangeably. Without loss of generality, following [39, 42], each dimension is normalized into the range $[0, 1]$ using min-max normalization.

User Preference. Following existing studies [9, 34, 36, 39, 42], we model the user's preference using a *linear function*, denoted by $f_{\boldsymbol{u}}(\cdot)$ and called *utility function*.

$$f_{\boldsymbol{u}}(\boldsymbol{p}) = \boldsymbol{u} \cdot \boldsymbol{p} = \sum_{i=1}^{d} u[i]p[i]$$

This function is parameterized by a non-negative vector $\boldsymbol{u} = (u[1], u[2], \ldots, u[d])$, namely *utility vector*, which captures the user's implicit preference weight for each dimension. A larger $u[i]$ signifies that the user places higher importance on the i-th dimension. Without loss of generality, following [39, 42], we assume $\sum_{i=1}^{d} u[i] = 1$. Function score $f_{\boldsymbol{u}}(\boldsymbol{p})$ is called the *utility* of $\boldsymbol{p}$. A higher utility indicates a stronger user preference for that point. Thus, for each dataset $\mathcal{D}_i$, the user's favorite point is $\boldsymbol{p}_i^* = \arg\max_{\boldsymbol{p} \in \mathcal{D}_i} f_{\boldsymbol{u}^*}(\boldsymbol{p})$, where $\boldsymbol{u}^*$ denotes the user's utility vector. The frequently used notations are summarized in our technical report [4].

Example 1. *Consider the points in Table 2. Assume that the underlying utility function is $f_{\boldsymbol{u}}(\boldsymbol{p}) = 0.4p[1] + 0.6p[2]$, i.e., the underlying utility vector is $\boldsymbol{u} = (0.4, 0.6)$. Consequently, the utility of point $\boldsymbol{p}_2$ w.r.t. $\boldsymbol{u}$ is calculated as $f_{\boldsymbol{u}}(\boldsymbol{p}_2) = 0.4 \times 0.3 + 0.6 \times 0.4 = 0.36$. The utilities of other candidate points can be derived similarly.*

Problem IMD. If the user's utility vector $\boldsymbol{u}^*$ is known, the system only needs to find the point with the highest utility in each dataset. However, in practice, as discussed in [27, 42], users may have difficulties specifying their utility vectors exactly. This drives us to learn $\boldsymbol{u}^*$ through user interaction. Following [36, 39, 42], we employ a widely used interactive framework.

The system proceeds in rounds. In each interactive round, there are three phases. (1) *Point Selection*: The system strategically chooses a pair of points $\boldsymbol{p}$ and $\boldsymbol{q}$ from the datasets based on the information gathered in previous rounds. Then, it asks a pairwise comparison question by presenting the user with this pair of points and asking the user to indicate the preferred one. (2) *Information Maintenance*: Based on the user's feedback ($\boldsymbol{p}$ is more preferred or $\boldsymbol{q}$ is more preferred), the system updates its maintained information to learn the user's preference. (3) *Stopping Condition*: If the collected information is sufficient to identify the user's favorite point within each dataset, the interaction terminates and the desired points are returned. Otherwise, the system proceeds to the next round. Formally, we are interested in the following problem.

Problem 1. ***Interactive Query over Multiple Datasets (IMD)*** *Given a set of m datasets $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$, the goal is to interactively identify the user's favorite point within each individual dataset $\mathcal{D}_i$ while minimizing the total number of questions asked.*

As far as we know, we are the first to study the IMD problem. The challenge of the IMD problem lies in how to design the three phases of the interactive framework for the multi-dataset setting. Existing interactive algorithms are developed for a single dataset, where the system selects questions, maintains information, and checks the stopping condition w.r.t. one dataset. They cannot be directly reused for the IMD problem. For point selection, a good question should be informative for multiple datasets rather than only one. For information maintenance, feedback from each question should be shared across datasets. For the stopping condition, the system must ensure that the favorite point in every dataset has been identified. Thus, the IMD problem requires a new design that jointly handles the three phases across multiple datasets.

The following presents a lower bound on the number of questions asked. Due to the lack of space, the proofs of some theorems/lemmas in this paper can be found in our technical report [4].

Theorem 1. *Given a set of m datasets* $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$ *where* $|\mathcal{D}_i| = n_i$. *Let* $N = \max_{i\in[1,m]}\{n_i\}$. *Any interactive algorithm that utilizes pairwise comparisons requires* $\Omega(\log_2 N)$ *interactive rounds to identify the user's favorite point in each dataset.*

4 Baseline

We first introduce some preliminaries. Then, we revisit the existing algorithms UH-Random [42], UH-Simplex [42], RH [39], and HD-PI [39] and show how to adapt them to our problem IMD.

4.1 Preliminaries

Existing works study the interactive query from the perspective of geometry. Let us introduce the relevant concepts.

Utility Space. As defined, the utility vector $\boldsymbol{u}$ is non-negative and normalized. Consequently, the domain of $\boldsymbol{u}$, called the *utility space* and denoted by $\mathcal{U} = \{\boldsymbol{u} \in \mathbb{R}^d_+ \mid \sum_{i=1}^d u[i] = 1\}$, forms a $(d-1)$-dimensional polytope. For instance, as illustrated in Figure 1, when $d = 3$, the utility space $\mathcal{U}$ corresponds to a triangle in space $\mathbb{R}^3_+$.

Hyper-planes and Half-spaces. For any pair of points $\boldsymbol{p}, \boldsymbol{q} \in \mathrm{D}$, as shown in Figure 1, they define a hyper-plane in space $\mathbb{R}^d_+$:

$$h_{\boldsymbol{p},\boldsymbol{q}} = \{\boldsymbol{u} \in \mathbb{R}^d_+ \mid \boldsymbol{u} \cdot (\boldsymbol{p} - \boldsymbol{q}) = 0\}.$$

This hyper-plane passes through the origin with its normal vector in the same direction as $(\boldsymbol{p} - \boldsymbol{q})$. It divides the utility space into two half-spaces, $h^+_{\boldsymbol{p},\boldsymbol{q}}$ and $h^-_{\boldsymbol{p},\boldsymbol{q}}$. The *positive half-space* $h^+_{\boldsymbol{p},\boldsymbol{q}}$ encompasses all utility vectors where $\boldsymbol{p}$ is preferred to $\boldsymbol{q}$ (i.e., $\boldsymbol{u} \cdot \boldsymbol{p} > \boldsymbol{u} \cdot \boldsymbol{q}$, denoted by $\boldsymbol{p} \succ \boldsymbol{q}$), whereas the *negative half-space* $h^-_{\boldsymbol{p},\boldsymbol{q}}$ contains those where $\boldsymbol{q}$ is preferred to $\boldsymbol{p}$ (i.e., $\boldsymbol{u} \cdot \boldsymbol{p} < \boldsymbol{u} \cdot \boldsymbol{q}$, denoted by $\boldsymbol{p} \prec \boldsymbol{q}$).

4.2 Algorithms

Existing algorithms, such as UH-Random [42], UH-Simplex [42], RH [39], and HD-PI [39], are designed for single-dataset $\mathcal{D}$ setting. Although they employ different strategies, their underlying mechanisms can be abstracted into a unified interactive framework.

The key part is how to learn the user's utility vector based on the user feedback during the interaction. Combining the concepts introduced above, existing algorithms interpret each user feedback as a geometric constraint in the utility space. Suppose that a user is presented with a pair of points $\boldsymbol{p}$ and $\boldsymbol{q}$.

Algorithm 1: Baseline-ExistingAlg

```
1 Input: Datasets D = {D_1, D_2, ..., D_m}
2 Output: The set of the user's favorite points S
3 S ← ∅;
4 foreach i ∈ [1, m] do
5     p_i^* ← ExistingAlg(D_i);
6     S ← S ∪ {p_i^*};
7 return S;
```

Lemma 1. *[37, 39] If a user indicates* $\boldsymbol{p} \succ \boldsymbol{q}$ *(resp.* $\boldsymbol{p} \prec \boldsymbol{q}$*) during the interaction, the user's utility vector must locate in* $h^+_{\boldsymbol{p},\boldsymbol{q}}$ *(resp.* $h^-_{\boldsymbol{p},\boldsymbol{q}}$*).*

Following Lemma 1, existing algorithms maintain two objects throughout the interaction. (1) a utility range $\mathcal{R} \subseteq \mathcal{U}$, which is guaranteed to contain the user's utility vector based on his/her feedback so far; and (2) a candidate set C, which contains all potential points that may be the user's favorite one. Initially, the utility range $\mathcal{R}$ is set to be the entire utility space $\mathcal{U}$, and the candidate set C is set to be the full dataset $\mathcal{D}$. Subsequently, they proceed in rounds. Each round involves the following three phases.

Point Selection. Based on the current utility range $\mathcal{R}$ and the candidate set C, existing algorithms select a pair of points $(\boldsymbol{p}, \boldsymbol{q})$, formulating as a question to interact with the user. Specifically, they all utilize the utility range $\mathcal{R}$ for point selection. UH-Random and UH-Simplex select those that are capable of achieving the highest utility w.r.t. at least a utility vector in $\mathcal{R}$ [42]. RH adopts a heuristic strategy by choosing the point pair whose hyper-plane is closest to the center of $\mathcal{R}$ [39]. HD-PI divides the $\mathcal{R}$ into multiple sub-ranges and selects the point pair whose hyper-plane divide these sub-ranges the most evenly [39].

Information Maintenance. Upon receiving the user's feedback $\boldsymbol{p} \succ \boldsymbol{q}$ (resp. $\boldsymbol{p} \prec \boldsymbol{q}$), following Lemma 1, the utility range is updated to be $\mathcal{R} \leftarrow \mathcal{R} \cap h^+_{\boldsymbol{p},\boldsymbol{q}}$ (resp. $\mathcal{R} \leftarrow \mathcal{R} \cap h^-_{\boldsymbol{p},\boldsymbol{q}}$). Since the utility range $\mathcal{R}$ contains the user's utility vector, if a point cannot be the one with the highest utility w.r.t. at least a utility vector in $\mathcal{R}$, it cannot be the user's favorite point, and thus, can be safely discarded from the candidate set C. Following this criterion, the candidate set is C refined based on the updated $\mathcal{R}$.

Stopping Condition. The interaction terminates when only a single point remains in the candidate set. This point is returned as the final output. If $|C| > 1$, another interactive round proceeds.

To establish baselines for our IMD problem, we adapt existing algorithms as follows. The main idea is to sequentially operate them across the datasets $\{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$. As outlined in Algorithm 1, we maintain a point set S to store the output. Initially, S is empty (line 3). For each dataset $\mathcal{D}_i$, we use an existing algorithm (e.g., UH-Random [42], UH-Simplex [42], RH [39], and HD-PI [39]) to operate on it (lines 4-5). Once the user's favorite point within $\mathcal{D}_i$ is identified, it is recorded in S, and the existing algorithm advances to the next dataset $\mathcal{D}_{i+1}$ (line 6). The process persists until the user's favorite points for all m datasets have been successfully retrieved, at which point the set S of recorded points is returned (line 7).

Theorem 2. *Take the algorithm UH-Simplex as an example. Given a collection of m datasets* $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$*, where each dataset* $\mathcal{D}_i$ *contains* n_i *points, the baseline approach adapting algorithm*

UH-Simplex identifies the user's favorite point across all datasets in $O\left(\sum_{i=1}^{m} n_i\right)$ interactive rounds in the worst case, and $O(deg_{max} \cdot \sum_{i=1}^{m} \sqrt[d]{n_i})$ on average, where deg_{max} is a parameter used to evaluate distribution of the datasets.

5 Shared Utility Range

Although sequentially applying existing algorithms over datasets is simple, it is highly inefficient. The main drawback is that it treats different datasets unrelated and discards all preference information learned from previous datasets. It neglects a characteristic of the IMD problem. Although datasets contain different points, they are queried by the same user, whose underlying utility vector remains fixed across all datasets. This suggests an important principle: the preference information learned from one dataset can be transferred to others. Specifically, each user feedback induces a half-space which acts as a constraint to refine the utility range, and thus, identify the user's utility vector. Since the user's utility vector is fixed, the same utility vector governs the user feedback across all datasets. Thus, the half-spaces obtained from one dataset are also valid for other datasets. Formally, we have Lemma 2.

Lemma 2. *If after processing some datasets, the system obtains a utility range $\mathcal{R}$ by intersecting the half-space induced by the user's feedback, then the user's utility vector lies in $\mathcal{R}$ and $\mathcal{R}$ can be safely used as the initial utility range for any unprocessed dataset.*

Motivated by this, we introduce the *shared utility range (SUR)* mechanism to enhance existing algorithms. The main idea is to maintain a global utility range $\mathcal{R}$ throughout the entire interaction process. When the system finishes processing a dataset, the resulting $\mathcal{R}$ is propagated to the next dataset as its initial utility range, instead of restarting from the entire utility space $\mathcal{U}$. This allows subsequent datasets to start from a smaller range, thereby avoiding questions that merely relearn redundant preference constraints.

Building upon the global utility range $\mathcal{R}$, the initialization of the candidate set can also be improved when moving to a new dataset during the interaction. Suppose that dataset $\mathcal{D}_i$ starts to be processed. Instead of initializing the candidate set as the entire dataset $\mathcal{D}_i$, we retain only those points that might be the user's favorite point. Specifically, a point $\boldsymbol{p} \in \mathcal{D}_i$ can be safely pruned if there does not exist any utility vector $\boldsymbol{u} \in \mathcal{R}$ under which $\boldsymbol{p}$ has the highest score among $\mathcal{D}_i$. Since the user's utility vector is guaranteed to reside within $\mathcal{R}$, such a point can never be the user's favorite one. Guided by this insight, we propose a sufficient condition to determine whether a point is qualified to be pruned.

Lemma 3. *If there exists a point $\boldsymbol{q} \in \mathcal{D}_i$ such that $\mathcal{R} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$, $\boldsymbol{p}$ cannot be the point with the highest utility vectors w.r.t. any $\boldsymbol{u} \in \mathcal{R}$.*

To efficiently verify whether $\mathcal{R} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$, we utilize the *extreme points* of $\mathcal{R}$ [11]. Here, the extreme points are the corner points of a polytope. For example, in Figure 1, (1, 0, 0), (0, 1, 0), and (0, 0, 1) are the extreme points of the utility space.

Lemma 4. *If all extreme points of $\mathcal{R}$ lie in $h^+_{\boldsymbol{q},\boldsymbol{p}}$, then $\mathcal{R} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$.*

To maximize the pruning efficiency, we introduce a *hard-to-easy* heuristic by pre-sorting the datasets in D in descending order based on their sizes. The intuition is that larger datasets contain more points and therefore induce a larger pool of candidate interactive questions. This gives more opportunities to select informative questions that can effectively shrink the utility range. Once a tighter global range is obtained, it can be propagated to the remaining datasets and benefit their candidate sets pruning.

Algorithm 2: SUR-ExistingAlg

```
1 Input: m datasets D = {D_1, D_2, ..., D_m};
2 Output: The set of the user's favorite points S;
3 Initialize the global utility range R ← U and set S ← ∅ ;
4 Sort D by the size of each D_i in descending order;
5 for i = 1 to m do
6     C_i ← D_i, and prune invalid points in C_i based on R_i;
7     p_i^*, R ← ExistingAlg(C_i, R);
8     S ← S ∪ {p_i^*};
9 return S;
```

The pseudocode of the existing algorithms enhanced with the shared utility range mechanism (SUR-UH-Random, SUR-UH-Simplex, SUR-RH, and SUR-HD-PI) is shown in Algorithm 2. In the beginning, we initialize the global utility range $\mathcal{R}$ as the entire utility space and the output set S to be empty (line 2). Then, the datasets are sorted in descending order based on their sizes (line 3). After sorting, the datasets are processed sequentially. Consider the i-th dataset $\mathcal{D}_i$. We create a candidate set C_i initialized to be $\mathcal{D}_i$, and refine it by using Lemma 3 and Lemma 4 to prune points based on $\mathcal{R}$ (line 6). After obtaining C_i, we apply existing algorithms on C_i with $\mathcal{R}$ as its initial utility range (line 7). Once the user's favorite point within $\mathcal{D}_i$ is identified, it is added to S, and the refined utility range returned by the existing algorithm is retained as the global utility range for subsequent datasets (line 8). This process continues until all datasets have been processed, after which S is returned (line 9).

Let us take the existing algorithm UH-Simplex as an example to analyze the number of questions asked to the user. Let $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$ be a set of datasets, sorted in descending order based on their sizes such that $n_1 \geq n_2 \geq \cdots \geq n_m$. Let $N = n_1$ be the maximum dataset size. Assuming that the points in each dataset are uniformly distributed in the space. Let $V(\mathcal{R}_{i-1})$ and $V(\mathcal{U})$ denote the volume of $\mathcal{R}_{i-1}$ and $\mathcal{U}$, respectively.

Theorem 3. *Suppose that all data points are uniformly distributed on the spherical surface, algorithm SUR-UH-Simplex identifies the user's favorite point within each dataset using at most $O(\sum_{i=1}^{m} n_{\mathcal{R}_{i-1}})$ interactive rounds in the worst case, and $O\left(deg_{max} \cdot \sum_{i=1}^{m} \sqrt[d]{n_{\mathcal{R}_{i-1}}}\right)$ rounds on average, where $n_{\mathcal{R}_{i-1}} = \lceil n_i \cdot \frac{V(\mathcal{R}_{i-1})}{V(\mathcal{U})} \rceil$ and deg_{max} is a parameter used to evaluate distribution of the datasets.*

6 Shared Questions

In this section, we present our algorithm SharedQ. We first introduce the holistic idea of selecting hyper-planes across the global spectrum of datasets. Subsequently, we propose optimization strategies to minimize the underlying computational burden.

6.1 Hyper-plane Selection Across Datasets

The previous algorithms (SUR-UH-Random, SUR-UH-Simplex, SUR-RH, and SUR-HD-PI) typically select hyper-planes based on a *single*

dataset. However, this selection may potentially influence *multiple* datasets. Specifically, consider an interactive round with a comparison question between $\boldsymbol{p}$ and $\boldsymbol{q}$. Once the user provides feedback, the corresponding half-space, e.g., $h^+_{\boldsymbol{p},\boldsymbol{q}}$, is used to refine the global utility range $\mathcal{R}$. Later, when moving to other datasets, the range $\mathcal{R}$ is used to initialize the candidate set via point pruning. Therefore, a question selected based on one dataset may have a global point pruning effect across multiple datasets. Noticing this limitation, we propose a mechanism termed *shared questions*. The idea is to select questions based on multiple datasets together, expecting the resulting global utility range $\mathcal{R}$ to benefit multiple datasets.

Nevertheless, selecting a question by considering multiple datasets is non-trivial. There are two issues that need to be addressed: (1) how to measure whether a question is informative, and (2) how to measure the question across multiple datasets.

For the first issue, we adopt an output-driven view. Recall that the goal of the IMD problem is to identify the user's favorite point in each dataset. Therefore, the informativeness of a question should be measured by how effectively it helps rule out candidate points that could be the output. To capture this, we use the notion of *partition*.

Definition 1 (Partition). *For a point $\boldsymbol{p}$ in dataset $\mathcal{D}_i$, we can delineate a unique range in $\mathcal{U}$, called partition and denoted by $\Theta_{\boldsymbol{p}}$. It contains all utility vector $\boldsymbol{u} \in \mathcal{U}$ under which $\boldsymbol{p}$ has the highest utility among $\mathcal{D}_i$, i.e., $\Theta_{\boldsymbol{p}} = \{\boldsymbol{u} \in \mathcal{U} \mid \forall \boldsymbol{q} \in \mathcal{D}_i \setminus \{\boldsymbol{p}\},\ f_{\boldsymbol{u}}(\boldsymbol{p}) > f_{\boldsymbol{u}}(\boldsymbol{q})\}$.*

Intuitively, for a point $\boldsymbol{p} \in \mathcal{D}_i$, if the intersection of the partition $\Theta_{\boldsymbol{p}}$ and $\mathcal{R}$ is empty, i.e., $\Theta_{\boldsymbol{p}} \cap \mathcal{R} = \emptyset$, there does not exist any utility vector $\boldsymbol{u} \in \mathcal{R}$ under which $\boldsymbol{p}$ has the highest utility among $\mathcal{D}_i$. Since the user's utility vector $\boldsymbol{u}^*$ is guaranteed to lie in $\mathcal{R}$, point $\boldsymbol{p}$ cannot have the highest utility w.r.t. $\boldsymbol{u}^*$ among $\mathcal{D}_i$. This means $\boldsymbol{p}$ cannot be the user's favorite point and thus can be safely ruled out.

To build the partition for a point $\boldsymbol{p} \in \mathcal{D}_i$, we utilize the half-spaces. For each point $\boldsymbol{q} \in \mathcal{D}_i \setminus \{\boldsymbol{p}\}$, we build a half-space $h^+_{\boldsymbol{p},\boldsymbol{q}}$. Then, the intersection of these half-spaces forms the partition, i.e.,

$$\Theta_{\boldsymbol{p}} = \left(\bigcap_{\boldsymbol{q} \in \mathcal{D}_i \setminus \{\boldsymbol{p}\}} h^+_{\boldsymbol{p},\boldsymbol{q}} \right) \cap \mathcal{U}$$

Lemma 5. *The intersection of these half-spaces with $\mathcal{U}$ contains all utility vectors $\boldsymbol{u} \in \mathcal{U}$ under which $\boldsymbol{p}$ has the highest utility among $\mathcal{D}_i$.*

Now consider a question with points $\boldsymbol{p}$ and $\boldsymbol{q}$, which induces the hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$. After the user answers this question, the utility range $\mathcal{R}$ will be refined to either $h^+_{\boldsymbol{p},\boldsymbol{q}} \cap \mathcal{R}$ or $h^-_{\boldsymbol{p},\boldsymbol{q}} \cap \mathcal{R}$. This refinement may make some partitions disjoint from the updated utility range (i.e., $\Theta_{\boldsymbol{p}} \cap \mathcal{R} = \emptyset$), meaning that the corresponding points can no longer be the final output and can thus be safely pruned. Hence, the informativeness of a question can be measured by the number of partitions that are eliminated from $\mathcal{R}$.

Nevertheless, since we do not know the user's answer before asking the question, the partitions in either $h^+_{\boldsymbol{p},\boldsymbol{q}}$ or $h^-_{\boldsymbol{p},\boldsymbol{q}}$ may be eliminated. A question that eliminates many partitions if the user prefers $\boldsymbol{p}$ to $\boldsymbol{q}$ may be ineffective if the user prefers $\boldsymbol{q}$ to $\boldsymbol{p}$. To ensure that the selected question is informative regardless of the user's feedback, we consider both possible answers and use the worst case as the guaranteed elimination effect. Specifically, for each dataset $\mathcal{D}_i$, with a slight abuse of notations, we use $|h^+_{\boldsymbol{p},\boldsymbol{q}}|$ and $|h^-_{\boldsymbol{p},\boldsymbol{q}}|$ to denote the numbers of partitions that lie in $h^+_{\boldsymbol{p},\boldsymbol{q}}$ and $h^-_{\boldsymbol{p},\boldsymbol{q}}$,

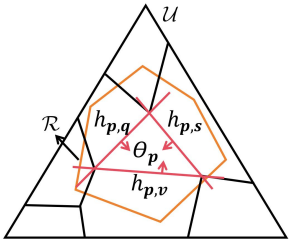


Figure 2: Partition and parameter range

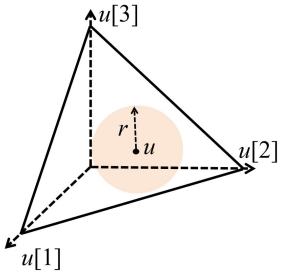


Figure 3: $\mathcal{B}(\boldsymbol{u}, r)$

respectively. The elimination effect of $h_{\boldsymbol{p},\boldsymbol{q}}$ on $\mathcal{D}_i$ is then measured by $E(h_{\boldsymbol{p},\boldsymbol{q}}, \mathcal{D}_i) = \min\{|h^+_{\boldsymbol{p},\boldsymbol{q}}|, |h^-_{\boldsymbol{p},\boldsymbol{q}}|\}$. This is a lower bound on the number of partitions that will be eliminated from $\mathcal{R}$ no matter how the user answers the question.

Example 2. *In Figure 4, within dataset $\mathcal{D}_1$, we observe that $\Theta_{\boldsymbol{p}_1} \subseteq h^+_{\boldsymbol{p},\boldsymbol{q}}$ and $\Theta_{\boldsymbol{p}_2}, \Theta_{\boldsymbol{p}_3}, \Theta_{\boldsymbol{p}_4} \subseteq h^-_{\boldsymbol{p},\boldsymbol{q}}$. Consequently, the score is evaluated as $E(h_{\boldsymbol{p},\boldsymbol{q}}, \mathcal{D}_1) = \min(|h^+_{\boldsymbol{p},\boldsymbol{q}}|, |h^-_{\boldsymbol{p},\boldsymbol{q}}|) = \min(1, 3) = 1$.*

For the second issue, we aggregate the elimination effects over all datasets. A question may make many candidates be eliminated in one dataset, but very few in another. Moreover, different datasets may have different numbers of points, and thus different numbers of partitions. To reflect this imbalance, we assign each dataset a weight according to the size of its current candidate set C_i: $W_i = |C_i|/(\sum_{j=1}^m |C_j|)$. Datasets with more points receive larger weights, because they require more effort to converge to a single one. Then, we define the *pruning score* as follows.

Definition 2 (Pruning Score). *The pruning score of a hyper-plane h is defined as $s(h) = \sum_{i=1}^m W_i \times E(h, \mathcal{D}_i)$, where $E(h_{\boldsymbol{p},\boldsymbol{q}}, \mathcal{D}_i) = \min\{|h^+_{\boldsymbol{p},\boldsymbol{q}}|, |h^-_{\boldsymbol{p},\boldsymbol{q}}|\}$ and $W_i = |C_i|/(\sum_{j=1}^m |C_j|)$.*

The pruning score connects multiple datasets in a unified way: each dataset contributes according to (1) how many points it has and (2) how many points the question can prune. We select the hyper-plane with the largest pruning score as the next question. In this way, the selected question is not merely useful for a single dataset, but provides the elimination benefit across multiple datasets.

The scoring function above formulates question selection as a global optimization problem, but computing it exactly is expensive. The cost mainly comes from two factors: the number of hyper-planes to be evaluated and the number of partitions to be checked for each hyper-plane. Consider m datasets, where each dataset $\mathcal{D}i$ contains n_i points. Since each pair of points may induce a candidate hyper-plane, the total number of hyper-planes is $O(\sum_{i=1}^m n_i^2)$. Moreover, each point may correspond to one output partition, so evaluating a hyper-plane requires checking its pruning effect over up to $O(\sum_{i=1}^m n_i)$ partitions across all datasets. Therefore, a direct implementation can be prohibitively costly. To address this issue, we develop two acceleration strategies: one reduces the number of hyper-planes considered, and the other reduces the number of candidate points examined during scoring.

6.2 Hyper-plane Reduction

The first acceleration strategy reduces the number of hyper-planes to be evaluated by the global scoring function. Let $\mathcal{H}_i$ denote the set of hyper-planes induced by point pairs in dataset $\mathcal{D}_i$. Instead of directly evaluating $s(h)$ for every hyper-plane $h \in \mathcal{H}_i$, we adopt a two-stage strategy. In the first stage, for each dataset $\mathcal{D}_i$, we select one representative hyper-plane h_i from its local pool $\mathcal{H}_i$. This step aims to find a hyper-plane that is likely to split the partitions effectively, without computing the pruning score for every hyper-plane in $\mathcal{H}_i$. In the second stage, we evaluate the pruning score only for the m representative hyper-planes $\{h_1, h_2, \ldots, h_m\}$ and choose the one with the largest score as the next question.

To select the representative hyper-plane h_i for each dataset, our idea is to use PCA [20, 33] to construct an approximate geometric reference and then choose the actual candidate hyper-plane that best matches this reference. For each point $\boldsymbol{p} \in C_i$, we denote the center of $\Theta_{\boldsymbol{p}} \cap \mathcal{R}$ by $\boldsymbol{u}_i(\boldsymbol{p})$. The set of centers $\{\boldsymbol{u}_i(\boldsymbol{p}) \mid \boldsymbol{p} \in C_i\}$ summarizes the distribution of partitions for dataset $\mathcal{D}_i$. We then apply PCA to these centers and obtain the first principal direction $\boldsymbol{w}_i$ with $|\boldsymbol{w}_i| = 1$. This direction captures the largest spread of the candidate partitions. A hyper-plane orthogonal to $\boldsymbol{w}_i$ is expected to separate the candidates effectively. We project all centers onto $\boldsymbol{w}_i$ and select the median of the projected centers as $\boldsymbol{u}_i$. These two components define a reference hyper-plane: $h_i' : \boldsymbol{w}_i \cdot (\boldsymbol{u} - \boldsymbol{u}_i) = 0$.

The ideal hyper-plane h_i' is only a geometric reference and may not correspond to a valid question, because valid questions must be induced by pairs of points and thus must come from $\mathcal{H}_i$. Therefore, we select the representative hyper-plane in $\mathcal{H}_i$ that best approximates h_i'. For a hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$, we define its difference from h_i' as

$$\mathrm{diff}(h_{\boldsymbol{p},\boldsymbol{q}}, h_i') = \alpha \left(1 - \frac{\boldsymbol{w}_i \cdot (\boldsymbol{p} - \boldsymbol{q})}{\|\boldsymbol{p} - \boldsymbol{q}\|}\right) + \beta \frac{|\boldsymbol{u}_i \cdot (\boldsymbol{p} - \boldsymbol{q})|}{\|\boldsymbol{p} - \boldsymbol{q}\|}$$

Here, the first term measures the angular deviation between the normal direction of h and the ideal direction $\boldsymbol{w}_i$, while the second term measures the perpendicular distance from the reference point $\boldsymbol{u}_i$ to h. The two parameters $\alpha, \beta > 0$ balance these two criteria. We then choose the hyper-plane that minimizes the *diff* score, i.e., $h_i = \arg\min_{h \in \mathcal{H}_i} \mathrm{diff}(h, h_i')$.

After obtaining one representative hyper-plane from each dataset, we compute the global score $s(h_i)$ for all $i \in 1, \ldots, m$ and select $h^* = \arg\max_{h_i \in \{h_1, \ldots, h_m\}} s(h_i)$. Thus, PCA is used only to reduce the hyper-plane search space locally, while the final question is still selected according to the cross-dataset pruning score. This preserves the global objective of shared questions while substantially reducing the number of hyper-planes that require expensive score evaluation.

Theorem 4. *Let v denote the maximum number of vertices among all partitions. The proposed PCA-based method reduces the time complexity of hyper-plane selection from $O\left(\left(\sum_{i=1}^m n_i^2\right) \cdot \left(\sum_{i=1}^m n_i\right) \cdot v\right)$ to $O\left(\sum_{i=1}^m n_i (d^2 + m \cdot v) + m \cdot d^3\right)$.*

6.3 Point Reduction

The second acceleration strategy is to reduce the number of points involved in C_i for each dataset $\mathcal{D}_i$. Our key idea is to localize the computation. Instead of considering the entire utility range $\mathcal{R}$, we maintain a localized utility range $\mathcal{R}_l \subseteq \mathcal{R}$ and only keep points whose partitions intersect $\mathcal{R}_l$. Since all subsequent questions are generated on these localized points, both the numbers of candidate partitions and candidate hyper-planes are significantly reduced.

Following this idea, we propose a strategy, namely *Detect-and-Divide*. At a high level, it repeatedly identifies a sub-range $\mathcal{R}_l$ of the current $\mathcal{R}$ and conducts two steps. The *detect* step finds the points whose output partitions intersect $\mathcal{R}_l$ and uses them to form localized candidate sets and hyper-plane pools. The *divide* step further refines $\mathcal{R}_l$ until there are only a few partitions intersecting $\mathcal{R}_l$. Together, these two steps replace global partition construction and hyper-plane enumeration with localized computation, substantially reducing the number of points examined in each round.

To achieve this, there are three objects maintained. (1) a sub-range $\mathcal{R}_l$, which is a sub-range of the current $\mathcal{R}$; (2) localized candidate sets $\mathrm{L} = \{\mathcal{L}_1, \mathcal{L}_2, \ldots, \mathcal{L}_m\}$, where $\mathcal{L}_i \subseteq \mathcal{D}_i$ stores the points whose partitions intersecting $\mathcal{R}_l$; and (3) localized hyper-plane sets $\mathrm{H} = \{\mathcal{H}_1, \mathcal{H}_2, \ldots, \mathcal{H}_m\}$, generated from point pairs within each $\mathcal{L}_i$.

Phase 1: Detect. The objective of the detect phase is to identify, for each dataset $\mathcal{D}_i$, all the partitions that intersect $\mathcal{R}_l$. The points associated with these partitions form the localized candidate set $\mathcal{L}_i$. Candidate hyper-planes are then generated only from points in $\mathcal{L}_i$, reducing the cost of subsequent question selection.

A straightforward approach is to construct the partition of every point in $\mathcal{D}_i$ and test whether it intersects $\mathcal{R}_l$. However, this requires examining the entire dataset and largely defeats the purpose of localization. To avoid such a global scan, we exploit the adjacency structure among output partitions.

Definition 3 (Partition Neighbors). *Two non-empty partitions $\Theta_{\boldsymbol{p}}$ and $\Theta_{\boldsymbol{q}}$ are called neighbors if they share a common boundary induced by the hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$. Correspondingly, their associated points $\boldsymbol{p}$ and $\boldsymbol{q}$ are called p-*neighbors.

The following lemma uses the concept of *partition neighbors* to provide the basis for discovering partitions through local expansion.

Lemma 6. *If a partition $\Theta_{\boldsymbol{p}} \neq \emptyset$ and $h_{\boldsymbol{p},\boldsymbol{q}}$ acts as a bounding hyper-plane of $\Theta_{\boldsymbol{p}}$, then $\Theta_{\boldsymbol{q}} \neq \emptyset$ and $\Theta_{\boldsymbol{q}}$ is adjacent to $\Theta_{\boldsymbol{p}}$ along $h_{\boldsymbol{p},\boldsymbol{q}}$.*

Lemma 6 implies that the neighbors of a partition can be identified from its bounding hyper-planes. Therefore, instead of constructing all partitions independently, we can start from one known partition and recursively visit its neighboring partitions. Specifically, for each dataset $\mathcal{D}_i$, we first find the point with the highest utility w.r.t. a utility vector in $\mathcal{R}_l$, i.e., $\boldsymbol{p}_i^0 = \arg\max_{\boldsymbol{p} \in \mathcal{D}_i} f_{\boldsymbol{u}}(\boldsymbol{p})$. Since $\boldsymbol{u} \in \mathcal{R}_l$, the partition of $\boldsymbol{p}_i^0$ contains $\boldsymbol{u}$ and therefore intersects $\mathcal{R}_l$. We use $\boldsymbol{p}_i^0$ as the anchor point and initialize $\mathcal{L}_i = \{\boldsymbol{p}_i^0\}$.

Starting from this anchor, we recursively explore neighboring partitions. For each unvisited point $\boldsymbol{p} \in \mathcal{L}_i$, we construct its partition and identify its bounding hyper-planes. By Lemma 6, every bounding hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$ identifies a neighboring partition $\Theta_{\boldsymbol{q}}$. The corresponding $\boldsymbol{q}$ is added to $\mathcal{L}_i$ only if its partition intersects the sub-range $\mathcal{R}_l$. The bounding hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}}$ is inserted into $\mathcal{H}_i$ for subsequent question generation. This expansion continues until all the points in $\mathcal{L}_i$ have been visited.

Example 3. *As illustrated in Figure 5, we initiate the process by randomly sampling a utility vector $\boldsymbol{u}$ to construct the localized hypersphere $\mathcal{B}(\boldsymbol{u}, r)$. Next, we identify the optimal point $\boldsymbol{p}_1$ with respect to $\boldsymbol{u}$ and insert it into the candidate set $\mathcal{L}_i$. Subsequently, we locate all*

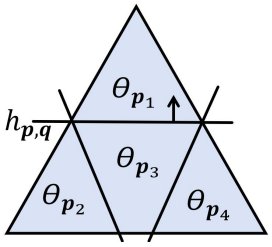


Figure 4: $E(h_{p,q}, \mathcal{D}_i)$

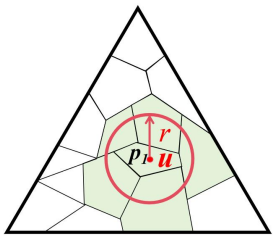


Figure 5: Detect: $\mathcal{R}_l$

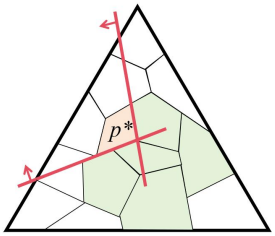


Figure 6: Divide: case 1

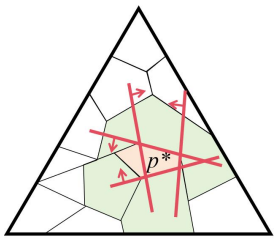


Figure 7: Divide: case 2

p-neighbors of $\boldsymbol{p}_1$ *by extracting the valid boundaries of its partition* $\Theta_{\boldsymbol{p}_1}$*. These neighboring partitions, depicted as the green regions in the figure, correspond to candidate points that are then appended to* $\mathcal{L}_i$*. Concurrently, their bounding hyper-planes are incorporated into the candidate hyper-plane set* $\mathcal{H}_i$*.*

Throughout the process, two issues remain. The first is how to define an appropriate sub-range $\mathcal{R}_l$, and the second is how to check if a partition intersects $\mathcal{R}_l$.

For the first issue, we define the sub-range as $\mathcal{R}_l = \mathcal{R} \cap \mathcal{B}(\boldsymbol{u}, r)$. The intersection with $\mathcal{R}$ ensures that every utility vector in $\mathcal{R}_l$ remains consistent with all user feedback collected so far. $\mathcal{B}(\boldsymbol{u}, r)$ is a hyper-sphere centered at $\boldsymbol{u}$ with radius r, where $\boldsymbol{u}$ is a utility vector randomly sampled from the current global utility range $\mathcal{R}$ or selected strategically, which will be discussed later.

The radius r determines the size of the localized range, and thus, controls the trade-off between computational efficiency and the number of partitions. A smaller radius restricts the search to fewer partitions, thereby reducing the number of points and hyper-planes involved. However, if the radius is too small, only a few partitions can be identified. The localized candidate sets may contain too few points to construct a valid comparison question. In contrast, a larger radius covers more candidate partitions but gradually approaches global processing and weakens the benefit of localization.

We therefore control the size of $\mathcal{R}_l$ by introducing a concept called *selection ratio* ρ, which represents the expected fraction of candidate partitions covered by the localized range. Assuming that candidate partitions are approximately uniformly distributed over the $(d-1)$-dimensional utility range, ρ can be approximated by the ratio between the volume of the local $(d-1)$-dimensional ball and that of the utility range. This yields the following result.

Lemma 7. *Assume that all data points are uniformly distributed on the spherical surface. Given a target selection ratio* ρ*, if we set the radius of the localized hyper-sphere as*

$$r = \left(\rho \cdot \frac{\sqrt{d}}{(d-1)!} \cdot \frac{\Gamma\left(\frac{d-1}{2} + 1\right)}{\pi^{(d-1)/2}} \right)^{\frac{1}{d-1}},$$

the expected number of partitions falling within the hyper-sphere is $\rho \cdot n_i$*, where* n_i *denote the size of* $\mathcal{D}_i$*.*

Using ρ instead of a fixed radius provides a dimension-aware way to control the expected number of partitions. In some cases, the resulting radius may be too small, causing every localized candidate set to contain only its anchor point and leaving no point pair from which to construct a question. When this occurs, we double r and re-detect. This process continues until at least one dataset produces a valid candidate hyper-plane.

For the second issue, we solve the following optimization problem to check if partition $\Theta_{\boldsymbol{q}}$ intersects with the hyper-sphere. Let $\mathcal{B}(\boldsymbol{u}_{\mathcal{B}}, r)$ be the hyper-sphere.

$$\begin{aligned} &\text{Minimize} && |\boldsymbol{u} - \boldsymbol{u}_{\mathcal{B}}|^2 \\ &\text{Subject to} && \boldsymbol{u} \in \Theta_{\boldsymbol{q}} \cap \mathcal{R}. \end{aligned}$$

This problem computes the smallest squared distance from the center $\boldsymbol{u}_{\mathcal{B}}$ to a utility vector in $\Theta\boldsymbol{q} \cap \mathcal{R}$. Let the optimal objective value be δ^2. If $\delta \le r$, then there is a utility vector in $\Theta\boldsymbol{q} \cap \mathcal{R}$ that lies inside the hyper-sphere, and hence $\Theta_{\boldsymbol{q}} \cap \mathcal{R}_l \neq \emptyset$..

However, carrying out this Quadratic Programming is time-consuming. Thus, we apply an inexpensive vertex-based test instead. Consider a partition $\Theta_{\boldsymbol{p}}$. If at least one of its extreme points lies inside $\mathcal{R}_l$, then the intersection $\Theta_{\boldsymbol{p}} \cap \mathcal{R}_l$ is non-empty, and $\boldsymbol{p}$ can be directly added to the localized candidate set $\mathcal{L}_i$. In this way, we can obtain a smaller candidate set in each iteration, but it will not affect the correctness of the final result.

Phase 2: Divide. With the localized candidate sets $\mathcal{L}_i$ and their corresponding hyper-plane sets $\mathcal{H}_i$ established, we turn to the *divide* phase. The goal here is to iteratively refine $\mathcal{R}_l$ until there are only a few partitions intersecting $\mathcal{R}_l$. In each interactive round, we select a hyper-plane based on the pruning score. Based on the user's feedback, we prune partitions and the corresponding points in $\mathcal{L}_i$, accordingly. This process continues until each $\mathcal{L}_i$ is reduced to exactly one point. Then, we evaluate the spatial location of each remaining point $\boldsymbol{p}_i \in \mathcal{L}_i$.

Lemma 8. *If* $\boldsymbol{p}_i$ *is the last one left in* $\mathcal{L}_i$ *and it corresponds to an internal partition in* $\mathcal{R}_l$*,* $\boldsymbol{p}_i$ *is the user's favorite point in this dataset.*

Proof. Suppose that there is another point $\boldsymbol{p}_i'$ that is user's true favorite point in dataset $\mathcal{D}_k$. Thus, we have $f_{\boldsymbol{u}}(\boldsymbol{p}_i') > f_{\boldsymbol{u}}(\boldsymbol{p}_i)$. If $\boldsymbol{p}$ corresponds to an internal partition in $\mathcal{L}_i$, according to the definition of partition, for any other point $\boldsymbol{p}_j \in \mathcal{D}_k / \{\boldsymbol{p}_i\}$, $f_{\boldsymbol{u}}(\boldsymbol{p}_i) > f_{\boldsymbol{u}}(\boldsymbol{p}_j)$, which is contradictory to $f_{\boldsymbol{u}}(\boldsymbol{p}_i) < f_{\boldsymbol{u}}(\boldsymbol{p}_i')$. Therefore, $\boldsymbol{p}_i$ is the only user's favorite point in $\mathcal{D}_k$. □

Based on Lemma 8, if the partition $\Theta_{\boldsymbol{p}}$ does not intersect the boundary of $\mathcal{R}_l$, we can confidently return $\boldsymbol{p}$ as the user's favorite point. Otherwise, boundary contact indicates that superior options may reside outside the current localized range, thus triggering a new *detect* round. Specifically, by the definition of the separating hyper-plane, each user feedback ($\boldsymbol{p} \succ \boldsymbol{q}$) in this round corresponds

Algorithm 3: Algorithm SharedQ

1 **Input:** A set of m datasets $\mathrm{D} = \{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_m\}$
2 **Output:** The user's favorite point $\boldsymbol{p}_i^*$ for each dataset $\mathcal{D}_i \in \mathrm{D}$
3 $\mathcal{R} \leftarrow \mathcal{U}$; $\boldsymbol{u}_{\mathcal{B}} \leftarrow$ a sampled utility vector in $\mathcal{R}$;
4 **for** $i = 1$ **to** m **do**
5 Done$(\mathcal{D}_i) \leftarrow$ **false**;
6 **while** ***true*** **do**
7 **for** $i = 1$ **to** m **do**
8 **if** *Done*$(\mathcal{D}_i) =$ ***false*** **then**
9 $\langle \mathcal{L}_i, \mathcal{H}_i \rangle \leftarrow$ **Detect**$(\mathcal{D}_i, \boldsymbol{u}_{\mathcal{B}})$;
10 $\mathrm{L} = \{\mathcal{L}_1, \mathcal{L}_2, \ldots, \mathcal{L}_m\}$, $\mathrm{H} = \{\mathcal{H}_1, \mathcal{H}_2, \ldots, \mathcal{H}_m\}$;
11 $\{\boldsymbol{p}_1^*, \ldots, \boldsymbol{p}_m^*\} \leftarrow$ **Divide**(L, H);
12 **for** $i = 1$ **to** m **do**
13 **if** $\boldsymbol{p}_i^*$ *is qualified to be returned* **then**
14 Done$(\mathcal{D}_i) \leftarrow$ **true**;
15 **if** $\forall i \in \{1, \ldots, m\}$, *Done*$(\mathcal{D}_i) =$ ***true*** **then**
16 **return** $\{\boldsymbol{p}_1^*, \boldsymbol{p}_2^*, \ldots, \boldsymbol{p}_m^*\}$;
17 **else**
18 Find a new utility vector $\boldsymbol{u}_{\mathcal{B}}$;

Detect$(\mathcal{D}_i, \boldsymbol{u})$
Find the point $\boldsymbol{p}_i \in \mathcal{D}_i$ with the highest utility w.r.t. $\boldsymbol{u}$;
Initialize $\mathcal{L}_i \leftarrow \{\boldsymbol{p}_i\}$ and $\mathcal{H}_i \leftarrow \emptyset$;
while *there is an unvisited point in* $\mathcal{L}_i$ **do**
  $\boldsymbol{p} \leftarrow$ unvisited point in $\mathcal{L}_i$;
  Identify neighboring partitions of $\Theta_{\boldsymbol{p}}$;
  Add the corresponding points into $\mathcal{L}_i$ if they are not included;
Construct $\mathcal{H}_i$ based on $\mathcal{L}_i$;
return $\langle \mathcal{L}_i, \mathcal{H}_i \rangle$;

Divide(L, H)
while $\exists i \in \{1, \ldots, m\}$ *s.t.* $|\mathcal{L}_i| > 1$ **do**
  Select the optimal hyper-plane $h_{\boldsymbol{p},\boldsymbol{q}} \in \mathrm{H}$;
  Present the question based on $(\boldsymbol{p}, \boldsymbol{q})$ to the user;
  Update $\mathcal{R}$ with the user's feedback;
  Prune invalid points in each $\mathcal{L}_i$;
return $\{\boldsymbol{p}_1^*, \ldots, \boldsymbol{p}_m^*\}$, *where* $\boldsymbol{p}_i^*$ *is the remaining point in* $\mathcal{L}_i$;

to a normal vector $(\boldsymbol{p} - \boldsymbol{q})$. We aggregate all these normal vectors to obtain a combined direction vector $\boldsymbol{v}$. Next, we calculate the center point $\boldsymbol{u}^*$ of the current sub-range $\mathcal{R}_l$. A ray is then projected from $\boldsymbol{u}^*$ along the direction of $\boldsymbol{v}$, intersecting the boundary of the global utility space $\mathcal{R}$ at a point denoted as $\boldsymbol{u}^{\mathcal{R}}$. Finally, we take the midpoint between $\boldsymbol{u}^*$ and $\boldsymbol{u}^{\mathcal{R}}$ as the new center for the hyper-sphere for the subsequent *detect and divide* round.

Example 4. *Suppose we have constructed the candidate set $\mathcal{L}_i$ and its corresponding hyper-plane set, as illustrated in Figure 5. After several rounds of user interaction, only a single point $\boldsymbol{p}^*$ remains in $\mathcal{L}_i$, as shown in Figure 6. We observe that the partition of $\boldsymbol{p}^*$ is located in the border of $\mathcal{R}_l$. Thus, we begin a new iteration using the method mentioned above. Conversely, in the alternative scenario depicted in Figure 7, the partition of surviving point $\boldsymbol{p}^*$ resides in the interior of $\mathcal{R}_l$. Therefore, we safely terminate the algorithm and return $\boldsymbol{p}^*$ as the user's favorite point.*

6.4 Summary

Combining the methodologies discussed above, the overall execution flow of the SharedQ framework is formalized in Algorithm 3. The algorithm initiates by setting the global utility range $\mathcal{R}$ to the entire space $\mathcal{U}$, sampling an initial anchor utility vector $\boldsymbol{u}_{\mathcal{B}}$, and initializing the resolution flag (*Done*) for each dataset to false (Lines 3–5). It then proceeds into a continuous loop to progressively isolate the user's true favorite points (Line 6). In each iteration, for every unresolved dataset $\mathcal{D}_i$, the algorithm invokes the Detect subprocedure to explore the local neighborhood around $\boldsymbol{u}_{\mathcal{B}}$, yielding a localized candidate set $\mathcal{L}_i$ and a candidate hyper-plane pool $\mathcal{H}_i$ (Lines 7–9). Subsequently, these local sets are aggregated into global collections L and H for the *divide* sub-procedure, which interactively refines $\mathcal{R}$ and prunes candidates until a single p_i^* remains (Lines 10–11). The algorithm evaluates whether each returned p_i^* satisfies the qualification criteria, marking the corresponding dataset as resolved if confirmed (Lines 12–14). Ultimately, if all m datasets achieve resolution simultaneously, the algorithm terminates and outputs the exact optimal points (Lines 15–16). Otherwise, it identifies a new utility vector $\boldsymbol{u}_{\mathcal{B}}$ to launch the subsequent iteration (Lines 17–18).

Theorem 5. *Assume that the points in each dataset are uniformly distributed in the space. Let V_{min} denote the minimum volume of any partition. If the detect phase is executed k times, algorithm SharedQ will identify the user's favorite point of each dataset within $O\left(k \log\left(\frac{r^d}{V_{min}}\right)\right)$ interactive rounds.*

7 Experiments

We conducted experiments on a machine with Intel(R) Xeon(R) Platinum 8259CL CPUs at 2.50GHz and 256GB RAM. All programs were complete in Python.

Datasets. We conduct our experiments on both synthetic and real-world datasets. The synthetic datasets include *correlated*, *independent*, and *anti-correlated* distributions, which are generated using the standard skyline dataset generator [8, 15, 21, 29]. Each dataset contains the same number of tuples. For the real datasets, we utilize the *Amazon Product* [3] and *Google Play Store* [1] datasets. We partition both real datasets by category, yielding 10 distinct sub-datasets for each source. The cardinality of these datasets varies significantly, ranging from $1,690$ to $309,391$ tuples, with dimensionalities ranging from 2 to 4. All attributes across all datasets are normalized into the range $(0, 1]$. Notably, we do not apply any skyline pre-computation to the raw data.

Algorithms. We evaluate our proposed algorithm UH-Random [42], UH-Simplex [42], RH [39], and HD-PI [39]. These baselines model the user's preference through a utility function, which is progressively learned via user interactions. Because these methods were not originally designed for our problem (i.e., identifying the best point for each dataset), we adapted them to ensure a fair comparison. UH-Random and UH-Simplex aim to find a point that minimizes a specific criterion known as the regret ratio [42], terminating the interaction when this ratio falls below a predefined threshold ϵ. To guarantee that the returned point is the user's exact favorite point, we set $\epsilon = 0$. Lastly, *RH* and HD-PI are traditionally

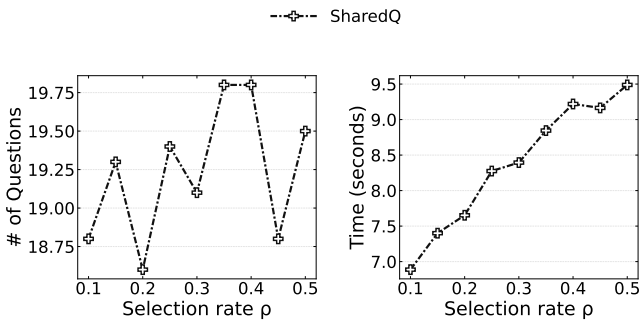


Figure 8: Parameter ρ

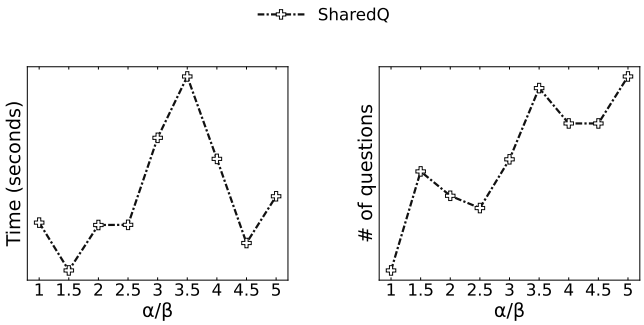


Figure 9: Parameter α/β

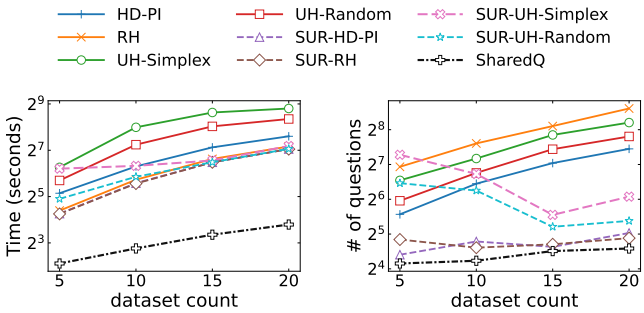


Figure 10: Different # of Datasets

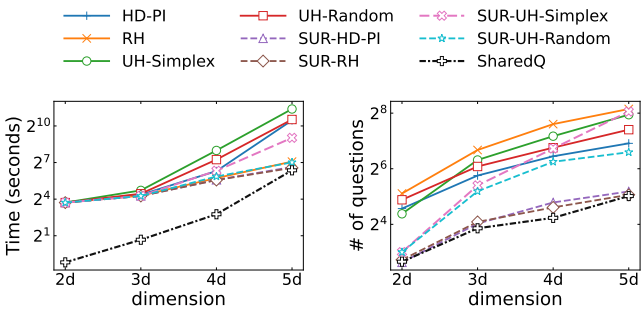


Figure 11: Vary d

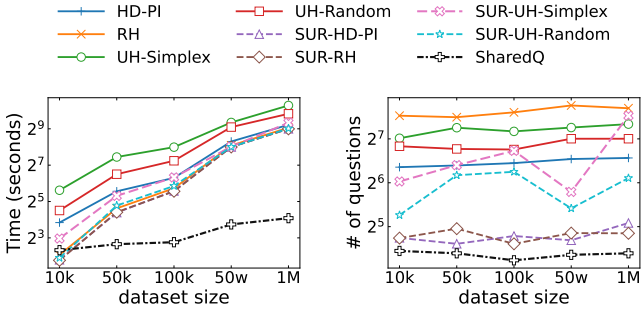


Figure 12: Different Size of Datasets

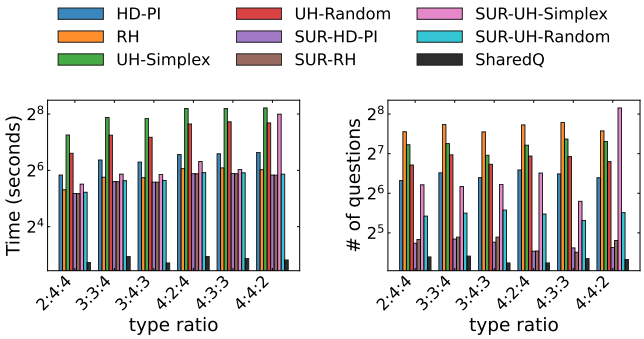
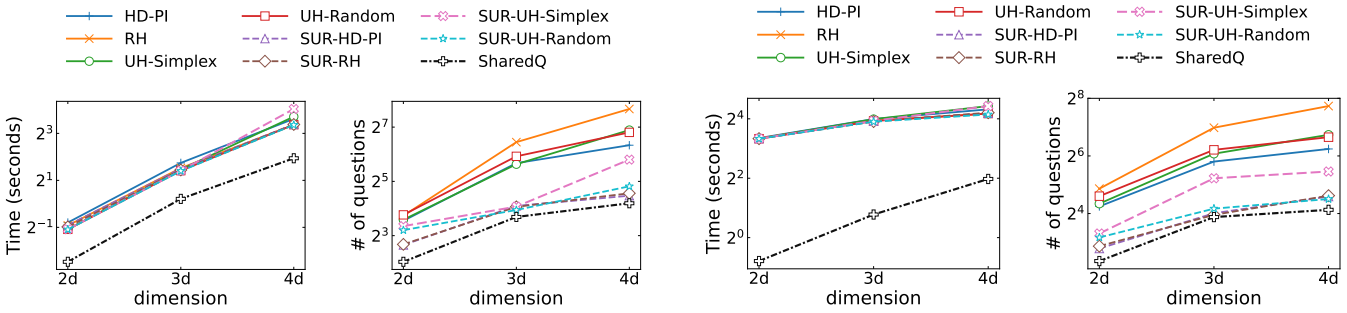


Figure 13: Different Type of Datasets

designed to return the user's top-k favorite points; to align with our formulation, we restrict its output by setting $k = 1$.

Parameter Setting. We evaluate the performance of our algorithms by varying several key parameters: (1) the selection ratio ρ, utilized in the DETECT algorithm (Section 6.3); (2) the parameters α and β, which measure the difference between two candidate hyper-planes (Section 6.2); (3) the number of datasets m; (4) the dimensionality d; (5) the size of each dataset N; and (6) the mixture proportions of the three synthetic data distributions (*correlated*, *anti-correlated*, and *independent*). Unless stated explicitly otherwise, the default parameter values for the synthetic datasets are set as $m = 10$, $N = 100,000$, and $d = 4$. Furthermore, the default data distribution mixture is $3 : 3 : 4$, corresponding to *correlated*, *anti-correlated*, and *independent* datasets, respectively.

Measurement. We evaluate the algorithms based on two primary measurements: (1) *Execution time*, defined as the total computational time required for the entire interactive process; and (2) *Number of questions*, which represents the total number of interaction rounds needed to successfully identify the user's favorite point in each dataset. To ensure reliability, we run each algorithm 10 times with different utility vectors and report the average results.



(a) Amazon (b) Google Playstore

Figure 14: Real Datasets

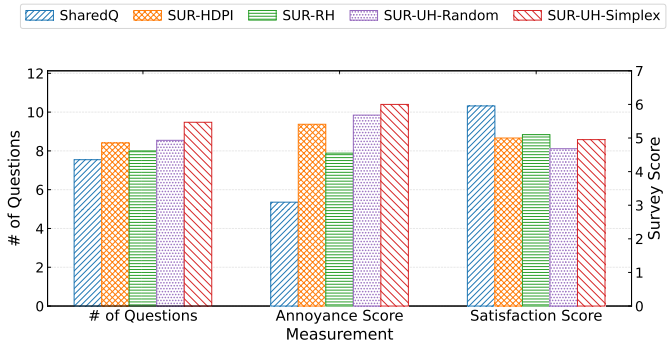


Figure 15: User Study Results

7.1 Performance Study of Our Algorithm

We first evaluate the impact of various parameters on the performance of SharedQ, focusing primarily on execution time and the number of questions asked. We begin by studying the selection ratio ρ. As illustrated in Figure 8, although setting $\rho = 0.2$ yields a slightly lower number of questions than $\rho = 0.1$, the latter exhibits a superior execution time. Taking this trade-off into account, we adopt $\rho = 0.1$ as the default setting for the remaining experiments.

Next, we evaluate the impact of parameters α and β. Based on the definition of the *diff* score, scaling α and β by the same constant does not alter the evaluation of any candidate hyper-plane; rather, it is their ratio that determines the outcome. Thus, we treat α/β as a unified parameter for this experiment. As shown in Figure 9, SharedQ achieves the minimum number of questions while maintaining a highly competitive execution time when $\alpha/\beta = 1$. Therefore, we set $\alpha = \beta = 1$ for all subsequent experiments.

7.2 Performance on Synthetic Datasets

We compare our SharedQ algorithm against four state-of-the-art interactive search algorithms (UH-Random [42], UH-Simplex [42], RH [39], and HD-PI [39]) as well as their corresponding variants enhanced with the *share utility range* (SUR) strategy. Unlike SharedQ, which entirely bypasses the need for skyline preprocessing, both the original baselines and their SUR variants require this step. To guarantee a fair comparison regarding execution time, we perform skyline pre-computation for all these competing algorithms and incorporate this preprocessing overhead into their total execution times.

The number of datasets m. Figure 10 evaluates the algorithms when varying the number of datasets m from 5 to 20, evenly distributed across *correlated*, *anti-correlated*, and *independent* distributions. While a larger m naturally introduces additional overhead, interaction costs scale much slower than execution time. As m grows, the performance gap between SUR-enhanced algorithms and sequential baselines widens significantly. Notably, our SharedQ algorithm consistently demonstrates absolute superiority across all settings of m, reducing the required questions by 50% to 87% and compressing total running time to under 1/10 compared to standard baselines.

The dimensionality d. In Figure 11, we study the performance of the algorithms as we vary the dimensionality d from 2 to 5. As expected, an increase in dimensionality inevitably leads to both prolonged execution times and higher interaction costs across all evaluated methods. However, our SharedQ algorithm consistently achieves significantly lower execution times when $d \leq 4$. It establishes a clear superiority when $d > 2$, reducing the interaction cost by 2 to 9 questions compared to SUR-HD-PI, and demonstrating an even more pronounced advantage over the original baselines. In terms of efficiency, while SharedQ maintains the shortest execution time across all dimensionalities, the performance margin narrows as d increases. This trend is primarily attributed to the fact that SharedQ requires computing the convex hull for the points in each dataset, a geometric operation whose computational complexity escalates exponentially with higher dimensionalities.

The size of each dataset N. Figure 12 illustrates the performance of the algorithms across varying dataset sizes N. The experimental results demonstrate the superiority of SharedQ, which effectively instantiates the intuition of restricting exploration to a localized subset within each dataset in a single round. By doing so, SharedQ bypasses the expensive computation of global partitions for all points. While this algorithmic advantage is less pronounced in smaller settings ($N = 10,000$), the execution time of SharedQ becomes significantly lower than that of all competing methods as the dataset scales to $N \geq 50,000$. Furthermore, rather than expanding

to a fixed number of partitions, SharedQ utilizes a localized hypersphere as the sub-range, which guarantees its robust performance consistency across different scales of N. In terms of interaction cost, compared to the standard baselines, SUR reduces the number of questions by nearly 65%, whereas SharedQ achieves an over 75% reduction.

The mixture proportions of datasets. Figure 13 evaluates the performance of the algorithms under varying mixtures of *correlated*, *anti-correlated*, and *independent* datasets. The efficiency of SUR is volatile, as it is governed by both the dataset processing sequence and the underlying spatial distribution of points. Because SUR propagates refined utility ranges sequentially, its pruning power depends heavily on whether early-explored datasets can offer high-quality constraints to prune more points before interaction. In contrast, algorithm SharedQ effectively mitigates these coupled limitations by globally optimizing shared hyperplanes across all datasets simultaneously. Thus, it demonstrates remarkable robustness and performance consistency across all distribution ratios, consistently outperforming all alternative approaches.

7.3 Performance on Real Dataset

We evaluate the performance of the four original baselines, their SUR variants, and our SharedQ algorithm on two real-world datasets: *Amazon Product* [3] and *Google Play Store* [1]. Both datasets are partitioned into subsets by category, resulting in varying sizes. We select 10 representative subsets to form the whole dataset for our evaluation, with dimensionalities ranging from 2 to 4. This real-world setup provides a highly heterogeneous environment, mirroring the diverse distribution mixtures evaluated in our synthetic datasets. As shown in Figure 14, while execution time and interaction costs naturally rise with dimensionality across all methods, SharedQ consistently demonstrates absolute superiority. Compared to the standard baselines, SharedQ significantly reduces the number of questions asked by 75% to 90%. Meanwhile, it achieves a massive 50% to 95% reduction in total execution time, highlighting its exceptional efficiency in practical scenarios.

7.4 User Study

To evaluate the practical usability of our framework and its resilience to real-world human factors (e.g., interaction fatigue and potential user errors), we conducted a user study utilizing a used car dataset [2]. Similar to our real-world dataset evaluations, we partitioned the dataset by the state of sale, retaining 1,000 tuples per subset. Each tuple is described by 4 attributes: price, year of purchase, odometer, and car condition. Following the setting in [30, 35, 41], we recruited 30 participants for this study and report their averaged results. Because our earlier experiments demonstrated that the four original baselines require an excessive number of interaction rounds—which would impose an unreasonable cognitive burden on participants—we restricted this user study to their SUR variants and our SharedQ algorithm.

We evaluate the algorithms based on three primary metrics: (1) The number of questions asked during the interaction. (2)Annoyance score: A subjective rating from 1 to 7 indicating the participant's level of fatigue or boredom during the interaction (1 denotes the least bored, and 7 denotes the most bored). (3)Satisfaction Score: A subjective rating from 1 to 7 reflecting the participant's satisfaction with the final returned tuples for each dataset (1 denotes the least satisfied, and 7 denotes the most satisfied).

Figure 15 presents the results of our user study. Algorithm SharedQ requires the fewest interactions, averaging only 7.5 questions per session, whereas the baseline SUR variants demand between 8.0 and 9.5 questions. Furthermore, SharedQ achieves the lowest Annoyance Score (3.1) and the highest Satisfaction Score (6.0). In contrast, all competing algorithms incur annoyance scores exceeding 4.6 and satisfaction scores below 5.1 (with SUR-RH performing the best among the baselines). These results clearly demonstrate that SharedQ delivers a vastly superior user experience by effectively mitigating interaction fatigue while maximizing user satisfaction.

7.5 Summary

The experimental evaluation demonstrates the superiority of our framework over best-known methods:(1) Efficiency and Effectiveness: Algorithm SharedQ achieves an exceptional performance by significantly minimizing both the interaction overhead and execution time compared to existing approaches (e.g. on the 4D dataset, while the standard baseline algorithms and their SUR variants require at least 87.3 and 24.4 questions, respectively, SharedQ successfully isolates the target points with merely 18.8 interactions).(2) Scalability: Our algorithm exhibits robust scalability with respect to both dataset size and the number of datasets. (e.g. in a environment with 20 distinct datasets, SharedQ maintains a minimal interaction cost of only 24.0 questions, whereas the baseline algorithms and their SUR variants demand at least 174.7 and 29.5 questions, respectively).(3) Real-World Applicability: Our framework delivers outstanding performance in practical deployment. (e.g. when evaluated on the 4D real-world *Google Play Store* dataset, Algorithm SharedQ achieves an over 75% reduction in the number of questions required compared to the traditional baseline solutions).

8 Conclusion

In this paper, we present an interactive framework designed to identify a user's exact favorite tuple over multiple datasets while minimizing the interaction workload. We first establish a baseline approach, which inherently suffers from prohibitive computational costs. To overcome this limitation, we introduce the optimization mechanisms of *shared utility ranges* and *shared questions*, culminating in the development of Algorithm SharedQ. By synergistically combining global utility space division with local candidate exploration, SharedQ effectively achieves both a minimal number of interaction rounds and significantly reduced system execution time. Furthermore, we formulate the optimal question generation as a set-cover problem and integrate a PCA-based heuristic to drastically reduce the computational overhead in each interactive round. The experiment results show that our algorithm reduce the number of question required for interaction and execution time significantly under typical settings. For future work, we plan to incorporate fault tolerance for user feedback errors across multiple datasets and extend our framework to handle heterogeneous environments where attributes differ across datasets.

References

[1] 2019. Google Play Store Apps. https://www.kaggle.com/datasets/gauthamp10/google-playstore-apps.

[2] 2021. Used Cars Dataset. https://www.kaggle.com/datasets/austinreese/craigslist-carstrucks-data.

[3] 2023. Amazon UK Products Dataset 2023. https://www.kaggle.com/datasets/asaniczka/amazon-uk-products-dataset-2023.

[4] Anonymous. 2026. *Interactive Query over Multiple Datasets.* Technical Report. https://github.com/Welt000/Interactive-Query-over-Multiple-Datasets

[5] Wolf-Tilo Balke, Ulrich Güntzer, and Christoph Lofi. 2007. Eliciting matters–controlling skyline sizes by incremental integration of user preferences. In *International Conference on Database Systems for Advanced Applications*. Springer, 551–562.

[6] Ilaria Bartolini, Paolo Ciaccia, and Marco Patella. 2014. Domination in the probabilistic world: Computing skylines for arbitrary correlations and ranking semantics. *ACM Transactions on Database Systems (TODS)* 39, 2 (2014), 1–45.

[7] Ilaria Bartolini, Paolo Ciaccia, Florian Waas, et al. 2001. FeedbackBypass: A new approach to interactive similarity query processing. In *VLDB*, Vol. 1. 201–210.

[8] Stephan Borzsony, Donald Kossmann, and Konrad Stocker. 2001. The skyline operator. In *Proceedings 17th international conference on data engineering*. IEEE, 421–430.

[9] Yuan-Chi Chang, Lawrence Bergman, Vittorio Castelli, Chung-Sheng Li, Ming-Ling Lo, and John R Smith. 2000. The onion technique: Indexing for linear optimization queries. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of data*. 391–402.

[10] Sean Chester, Alex Thomo, S Venkatesh, and Sue Whitesides. 2014. Computing k-regret minimizing sets. *Proceedings of the VLDB Endowment* 7, 5 (2014), 389–400.

[11] Mark De Berg, Otfried Cheong, Marc Van Kreveld, and Mark Overmars. 2008. *Computational geometry: Algorithms and applications*. Springer Berlin Heidelberg.

[12] Ting Deng and Wenfei Fan. 2013. On the complexity of query result diversification. *Proceedings of the VLDB Endowment (PVLDB)* 6, 8 (2013), 577–588.

[13] Eyal Dushkin and Tova Milo. 2018. Top-k sorting under partial order information. In *Proceedings of the 2018 International Conference on Management of Data*. 1007–1019.

[14] Brian Eriksson. 2013. Learning to top-k search using pairwise comparisons. In *Artificial Intelligence and Statistics*. PMLR, 265–273.

[15] Yunjun Gao, Qing Liu, Baihua Zheng, Li Mou, Gang Chen, and Qing Li. 2015. On processing reverse k-skyband and ranked reverse skyline queries. *Information Sciences* 293 (2015), 11–34.

[16] Ihab F Ilyas, George Beskales, and Mohamed A Soliman. 2008. A survey of top-k query processing techniques in relational database systems. *ACM Computing Surveys (CSUR)* 40, 4 (2008), 1–58.

[17] Kevin G Jamieson and Robert Nowak. 2011. Active ranking using pairwise comparisons. *Advances in neural information processing systems* 24 (2011).

[18] Yiling Jia, Huazheng Wang, Stephen Guo, and Hongning Wang. 2021. Pairrank: Online pairwise learning to rank by divide-and-conquer. In *Proceedings of the web conference 2021*. 146–157.

[19] Bin Jiang, Jian Pei, Xuemin Lin, David W Cheung, and Jiawei Han. 2008. Mining preferences from superior and inferior examples. In *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*. 390–398.

[20] Ian T Jolliffe and Jorge Cadima. 2016. Principal component analysis: a review and recent developments. *Philosophical transactions of the royal society A: Mathematical, Physical and Engineering Sciences* 374, 2065 (2016), 20150202.

[21] Georgia Koutrika, Evaggelia Pitoura, and Kostas Stefanidis. 2013. Preference-based query personalization. In *Advanced Query Processing: Volume 1: Issues and Trends*. Springer, 57–81.

[22] Jongwuk Lee, Gae-won You, Seung-won Hwang, Joachim Selke, and Wolf-Tilo Balke. 2012. Interactive skyline queries. *Information Sciences* 211 (2012), 18–35.

[23] Tie-Yan Liu et al. 2009. Learning to rank for information retrieval. *Foundations and Trends® in Information Retrieval* 3, 3 (2009), 225–331.

[24] Lucas Maystre and Matthias Grossglauser. 2017. Just sort it! A simple and effective approach to active preference learning. In *International Conference on Machine Learning*. PMLR, 2344–2353.

[25] Denis Mindolin and Jan Chomicki. 2009. Discovering relative importance of skyline attributes. *Proceedings of the VLDB Endowment* 2, 1 (2009), 610–621.

[26] Baharan Mirzasoleiman, Amin Karbasi, Rik Sarkar, and Andreas Krause. 2013. Distributed submodular maximization: Identifying representative elements in massive data. *Advances in Neural Information Processing Systems* 26 (2013).

[27] Danupon Nanongkai, Ashwin Lall, Atish Das Sarma, and Kazuhisa Makino. 2012. Interactive regret minimization. In *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*. 109–120.

[28] Danupon Nanongkai, Atish Das Sarma, Ashwin Lall, Richard J Lipton, and Jun Xu. 2010. Regret-minimizing representative databases. *Proceedings of the VLDB Endowment* 3, 1-2 (2010), 1114–1124.

[29] Dimitris Papadias, Yufei Tao, Greg Fu, and Bernhard Seeger. 2005. Progressive skyline computation in database systems. *ACM Transactions on Database Systems (TODS)* 30, 1 (2005), 41–82.

[30] Li Qian, Jinyang Gao, and HV Jagadish. 2015. Learning user preferences by adaptive pairwise comparison. *Proceedings of the VLDB Endowment* 8, 11 (2015), 1322–1333.

[31] R Tyrrell Rockafellar. 1997. *Convex analysis*. Vol. 28. Princeton university press.

[32] Gerard Salton. 1989. Automatic text processing: The transformation, analysis, and retrieval of. *Reading: Addison-Wesley* 169 (1989).

[33] Jonathon Shlens. 2014. A tutorial on principal component analysis. *arXiv preprint arXiv:1404.1100* (2014).

[34] Bo Tang, Kyriakos Mouratidis, and Man Lung Yiu. 2017. Determining the impact regions of competing options in preference space. In *Proceedings of the 2017 ACM International Conference on Management of Data*. 805–820.

[35] Yufei Tao, Jun Zhang, Dimitris Papadias, and Nikos Mamoulis. 2004. An efficient cost model for optimization of nearest neighbor search in low and medium dimensional spaces. *IEEE Transactions on Knowledge and Data Engineering* 16, 10 (2004), 1169–1184.

[36] Weicheng Wang, Victor Junqiu Wei, Min Xie, Di Jiang, Lixin Fan, and Haijun Yang. 2025. Interactive Search with Reinforcement Learning. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 1443–1455.

[37] Weicheng Wang and Raymond Chi-Wing Wong. 2022. Interactive mining with ordered and unordered attributes. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2504–2516.

[38] Weicheng Wang, Raymond Chi-Wing Wong, HV Jagadish, and Min Xie. 2024. Reverse regret query. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4100–4112.

[39] Weicheng Wang, Raymond Chi-Wing Wong, and Min Xie. 2021. Interactive search for one of the top-k. In *Proceedings of the 2021 International Conference on Management of Data*. 1920–1932.

[40] Min Xie Weicheng Wang, Raymond Chi-Wing Wong. 2023. Interactive Search with Mixed Attributes. In *In IEEE ICDE International Conference on Data Engineering*.

[41] Min Xie, Tianwen Chen, and Raymond Chi-Wing Wong. 2019. Findyourfavorite: An interactive system for finding the user's favorite tuple in the database. In *Proceedings of the 2019 International Conference on Management of Data*. 2017–2020.

[42] Min Xie, Raymond Chi-Wing Wong, and Ashwin Lall. 2019. Strongly truthful interactive regret minimization. In *Proceedings of the 2019 International Conference on Management of Data*. 281–298.

[43] Min Xie, Raymond Chi-Wing Wong, Jian Li, Cheng Long, and Ashwin Lall. 2018. Efficient k-Regret Query Algorithm with Restriction-free Bound for any Dimensionality. In *Proceedings of the 2018 International Conference on Management of Data*. Association for Computing Machinery, 959–974.

[44] Kai Yao, Jianjun Li, Guohui Li, and Changyin Luo. 2016. Efficient group top-k spatial keyword query processing. In *Asia-Pacific Web Conference*. Springer, 153–165.

[45] Meike Zehlike, Francesco Bonchi, Carlos Castillo, Sara Hajian, Mohamed Megahed, and Ricardo Baeza-Yates. 2017. Fa* ir: A fair top-k ranking algorithm. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1569–1578.

[46] Guangyi Zhang, Nikolaj Tatti, and Aristides Gionis. 2023. Finding favourite tuples on data streams with provably few comparisons. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3229–3238.

[47] Jiping Zheng and Chen Chen. 2020. Sorting-based interactive regret minimization. In *Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data*. Springer, 473–490.

[48] Kai Zheng, Han Su, Bolong Zheng, Shuo Shang, Jiajie Xu, Jiajun Liu, and Xiaofang Zhou. 2015. Interactive top-k spatial keyword queries. In *2015 IEEE 31st international conference on data engineering*. IEEE, 423–434.

Table 3: Frequently Used Notations

Notation	Definition
D and $\mathcal{D}_i$	All datasets and the i-th dataset.
m and n_i	Number of datasets and points in $\mathcal{D}_i$.
$\boldsymbol{p}$ and d	A point and dimensionality.
$\boldsymbol{u}$ and $\mathcal{R}$	Utility vector and global utility range.
$\Theta_{\boldsymbol{p}}$	Partition of point $\boldsymbol{p}$.
$h_{\boldsymbol{p},\boldsymbol{q}}$	Hyper-plane of $\boldsymbol{p}$ and $\boldsymbol{q}$.
$h^+_{\boldsymbol{p},\boldsymbol{q}}/h^-_{\boldsymbol{p},\boldsymbol{q}}$	Positive/negative half-space of $h_{\boldsymbol{p},\boldsymbol{q}}$.

A Table

The frequently used notations are shown in Table 3.

B Proof

Proof of Theorem 1. For any individual dataset $\mathcal{D}_i$, any algorithm driven by pairwise comparison can be modeled as a binary tree. Each leaf corresponds to a point, and each internal node corresponds to a question asked to a user. Since $\mathcal{D}_i$ contains n_i points, the decision tree must have at least n_i leaves, yielding a height of $\Omega(\log_2 n_i)$ to find the desired point. To complete the query task over all m datasets, the algorithm is lower-bounded by the largest dataset. Even assuming perfect parallelism across datasets, the total interaction rounds cannot be fewer than the maximum height among all individual trees, which is $\Omega(\max_{i\in[1,m]} \log_2 n_i) = \Omega(\log_2 N)$. □

Proof of Theorem 2. It has been proven that identifying a user's favorite point within a single dataset of size n in $O(n)$ interactive rounds in worst case and $O(\deg_{\max} \cdot \sqrt[d]{n})$ on average for algorithm UH-Simplex [42]. Since the baseline method processes each of the m datasets independently and sequentially without any information transfer, the total number of questions is bounded by the sum of their individual maximum requirements. Therefore, the overall upper bound is derived as:

$$\begin{cases} O\left(\sum_{i=1}^{m} n_i\right) & \text{worst case} \\ O\left(\deg_{\max} \cdot \sum_{i=1}^{m} \sqrt[d]{n_i}\right) & \text{on average} \end{cases}$$

□

Proof of Lemma 2. Each user feedback compares two points and, by Lemma 1, induces a half-space that contains the user's utility vector $\boldsymbol{u}^*$. Since $\mathcal{R}$ is obtained by intersecting such half-spaces, we have $\boldsymbol{u}^* \in \mathcal{R}$. The utility vector $\boldsymbol{u}^*$ is shared across all datasets in the IMD problem. Therefore, every half-space collected from a dataset is also valid for other datasets, and $\mathcal{R}$ can be safely reused. □

Proof of Lemma 3. According to the definition of $h_{\boldsymbol{q},\boldsymbol{p}}$, we have $f_{\boldsymbol{u}}(\boldsymbol{q}) > f_{\boldsymbol{u}}(\boldsymbol{p})$ for any $\boldsymbol{u} \in h^+_{\boldsymbol{q},\boldsymbol{p}}$. Given that $\mathcal{R} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$, any utility vector $\boldsymbol{u} \in \mathcal{R}$ must inherently reside within $h^+_{\boldsymbol{q},\boldsymbol{p}}$, which translates to $f_{\boldsymbol{u}}(\boldsymbol{q}) > f_{\boldsymbol{u}}(\boldsymbol{p})$. Consequently, the utility of $\boldsymbol{p}$ can never surpass that of $\boldsymbol{q}$ for any utility vector within $\mathcal{R}$. □

Proof of Lemma 4. Since $\mathcal{R}$ is formed by the intersection of several half-spaces, it is *convex* [11]. Let $\mathcal{E} = \{\boldsymbol{e}_1, \boldsymbol{e}_2, \ldots, \boldsymbol{e}_k\}$ denote the finite set of all extreme points of $\mathcal{R}$, where $\boldsymbol{e}_i \in h^+_{\boldsymbol{q},\boldsymbol{p}}$ holds for all $\boldsymbol{e}_i \in \mathcal{E}$. Since $\mathcal{R}$ is convex, according to the Krein-Milman theorem [31], any utility vector $\boldsymbol{u} \in \mathcal{R}$ can be represented as a convex combination of its extreme points $\mathcal{E}$. That is, there exist non-negative coefficients $\lambda_1, \lambda_2, \ldots, \lambda_k$ such that:

$$\boldsymbol{u} = \sum_{j=1}^{k} \lambda_j \boldsymbol{e}_j, \quad \text{where } \sum_{j=1}^{k} \lambda_j = 1 \text{ and } \lambda_j \geq 0. \tag{1}$$

Consequently, for any $\boldsymbol{u} \in \mathcal{R}$, the formula can be expanded as:

$$\boldsymbol{u} \cdot (\boldsymbol{q} - \boldsymbol{p}) = \left(\sum_{j=1}^{k} \lambda_j \boldsymbol{e}_j\right) \cdot (\boldsymbol{q} - \boldsymbol{p}) = \sum_{j=1}^{k} \lambda_j \left[\boldsymbol{e}_j \cdot (\boldsymbol{q} - \boldsymbol{p})\right].$$

Given that $\boldsymbol{e}_j \in h^+_{\boldsymbol{q},\boldsymbol{p}}$ (i.e., $\boldsymbol{e}_j \cdot (\boldsymbol{q} - \boldsymbol{p}) > 0$) and $\lambda_j \geq 0$ for all j, with at least one $\lambda_j > 0$, it follows that:

$$\boldsymbol{u} \cdot (\boldsymbol{q} - \boldsymbol{p}) > 0,$$

which implies $\boldsymbol{u} \in h^+_{\boldsymbol{q},\boldsymbol{p}}$. Since this relation holds universally for all $\boldsymbol{u} \in \mathcal{R}$, we successfully establish the conclusion $\mathcal{R} \subseteq h^+_{\boldsymbol{q},\boldsymbol{p}}$. □

Proof of Theorem 3. Prior work establishes that the UH-Simplex algorithm guarantees an interactive upper bound of $O(n)$ in the worst case, and $O(\deg_{\max} \cdot \sqrt[d]{n})$ on average, for a single dataset of size n [42]. Under the assumption that all data points are uniformly distributed on the spherical surface, when a refined utility range $\mathcal{R}_{i-1}$ is propagated to $\mathcal{D}_i$ as a global constraint, the expected number of surviving candidate points in $\mathcal{D}_i$ can be estimated by the volume ratio as $n_{\mathcal{R}_{i-1}} = \lceil n_i \cdot \frac{V(\mathcal{R}_{i-1})}{V(\mathcal{U})} \rceil$. Since the initial utility range $\mathcal{R}_0$ is defined as the entire space $\mathcal{U}$, we naturally have $n_{\mathcal{R}_0} = n_1$. By sequentially aggregating the interactive overhead across all datasets, we derive the overall complexity of the SUR-UH-Simplex algorithm as:

$$\begin{cases} O\left(\sum_{i=1}^{m} n_{\mathcal{R}_{i-1}}\right) & \text{worst case} \\ O\left(\deg_{\max} \cdot \sum_{i=1}^{m} \sqrt[d]{n_{\mathcal{R}_{i-1}}}\right) & \text{average case} \end{cases}$$

□

Proof of Theorem 4. As established in Section 6.2, the m datasets collectively generate $O(\sum_{i=1}^{m} n_i^2)$ candidate hyper-planes and $O(\sum_{i=1}^{m} n_i)$ partitions. In the conventional approach, one must exhaustively evaluate the positional relationship between every hyper-plane and every partition. According to [37], verifying this relationship requires checking all v extreme points of a partition/Consequently, the overall complexity of the traditional approach is:

$$O\left(\left(\sum_{i=1}^{m} n_i^2\right) \cdot \left(\sum_{i=1}^{m} n_i\right) \cdot v\right)$$

.

In our PCA-based method, computing the middle point of each partition requires $O(\sum_{i=1}^{m} n_i \cdot v)$ operations. Centering these points and calculating their mean is achieved in $O(\sum_{i=1}^{m} n_i \cdot d)$, while computing the covariance matrix introduces a complexity of $O(\sum_{i=1}^{m} n_i \cdot d^2)$. Next, the eigenvalue decomposition adds a bound of $O(d^3)$ per dataset, yielding an overall cost of $O(m \cdot d^3)$ for this step. Ultimately, evaluating the positional relationship of the final m optimal hyper-planes against all $O(\sum_{i=1}^{m} n_i)$ partitions necessitates $O(\sum_{i=1}^{m} n_i \cdot m \cdot v)$ operations. Summing these individual bounds, the overall time complexity of our PCA method reduces to:

$$O\left(\sum_{i=1}^{m} n_i(d^2 + m \cdot v) + m \cdot d^3\right).$$

□

Proof of Lemma 5. For any utility vector $\boldsymbol{u} \in \Theta_{\boldsymbol{p}} \cap \mathcal{U}$, suppose that there exists a point $\boldsymbol{q} \in \mathcal{D}_i \setminus \{\boldsymbol{p}\}$ whose utility is strictly higher that that of $\boldsymbol{p}$, i.e., $\boldsymbol{u} \cdot \boldsymbol{q} > \boldsymbol{u} \cdot \boldsymbol{p}$. By the definition of the partition, the premise $\boldsymbol{u} \in \Theta_{\boldsymbol{p}}$ implies that $\boldsymbol{u} \in h^+_{\boldsymbol{p},\boldsymbol{q}}$, which dictates $\boldsymbol{u} \cdot \boldsymbol{p} > \boldsymbol{u} \cdot \boldsymbol{q}$. This contradicts our assumption. Therefore, we can conclude that $\boldsymbol{p}$ yields the highest utility in $\mathcal{D}_i$ w.r.t. any $\boldsymbol{u} \in \Theta_{\boldsymbol{p}} \cap \mathcal{R}$. □

Proof of Lemma 6. By definition, for any $\boldsymbol{u} \in h^+_{\boldsymbol{p},\boldsymbol{q}}$, we have $f_{\boldsymbol{u}}(\boldsymbol{p}) > f_{\boldsymbol{u}}(\boldsymbol{q})$. Let $\boldsymbol{u'}$ be an arbitrary point lying on the boundary of $\Theta_{\boldsymbol{q}}$, we have $f_{\boldsymbol{u'}}(\boldsymbol{p}) = f_{\boldsymbol{u'}}(\boldsymbol{q})$. This implies that $\boldsymbol{u'}$ is also lies in the boundary of $\Theta_{\boldsymbol{q}}$, rendering $\Theta_{\boldsymbol{q}}$ non-empty. □

Proof of Lemma 7. Given that the utility vectors satisfy $\sum_{i=1}^{d} u_i = 1$ and $u_i \geq 0$, the global utility space is essentially a $(d-1)$-dimensional standard simplex embedded in $\mathbb{R}^d$. The $(d-1)$-dimensional volume of this utility space is known as $V_U = \frac{\sqrt{d}}{(d-1)!}$. Accordingly, the local neighborhood $\mathcal{B}(\boldsymbol{u}, r)$ restricted to this simplex is a $(d-1)$-dimensional hyper-ball, whose volume is given by $V_{\mathcal{B}}(r) = \frac{\pi^{(d-1)/2}}{\Gamma(\frac{d-1}{2}+1)} r^{d-1}$.

Under the assumption that all data points are uniformly distributed on the spherical surface, the volume of all partitions are the same. Thus, the expected selection ratio ρ is equivalent to the ratio of the volume of the local subset to the total volume of the utility space, i.e., $\rho = \frac{V_{\mathcal{B}}(r)}{V_U}$. By expanding this equation, we have:

$$\rho = \frac{\frac{\pi^{(d-1)/2}}{\Gamma(\frac{d-1}{2}+1)} r^{d-1}}{\frac{\sqrt{d}}{(d-1)!}}$$

Solving for r yields the desired approximation:

$$r = \left(\rho \cdot \frac{\sqrt{d}}{(d-1)!} \cdot \frac{\Gamma\left(\frac{d-1}{2}+1\right)}{\pi^{(d-1)/2}} \right)^{\frac{1}{d-1}}.$$

In this way, the expected number of partitions falling within the hyper-sphere $n_{\mathcal{B}}$ satisfies $\rho = \frac{n_{\mathcal{B}}}{n_i}$, i.e. $n_{\mathcal{B}} = \rho \cdot n_i$. □

Proof of Theorem 5. During the Detect phase, the algorithm identifies all candidate partitions enclosed within the localized hyper-sphere $\mathcal{B}(\boldsymbol{u}, r)$. Let $V_d(r) \propto r^d$ denote the volume of a d-dimensional hyper-sphere. Given the minimum partition volume V_{min}, the maximum number of candidate partitions within $\mathcal{B}(\boldsymbol{u}, r)$ is bounded by $O\left(\frac{r^d}{V_{min}}\right)$. Subsequently, in the Divide phase, the strategically selected hyper-plane efficiently eliminates approximately half of the candidate partitions in each interactive round. Consequently, the number of questions required to isolate a single point in one stage is bounded by $O\left(\log\left(\frac{r^d}{V_{min}}\right)\right)$. Accumulating this overhead across k stages, the total number of interactive rounds is given by:

$$R = O\left(k \log\left(\frac{r^d}{V_{min}}\right)\right).$$

□